```
// 源文件：android/AndroidStudioProject/app/src/main/java/cn/edu/nb3/myclass/MainActivity.kt
package cn.edu.nb3.myclass

import android.Manifest
import android.app.Activity
import android.content.ClipboardManager
import android.content.Context
import android.content.Intent
import android.content.SharedPreferences
import android.content.pm.PackageManager
import android.content.pm.ActivityInfo
import android.content.res.Configuration
import android.database.Cursor
import android.graphics.Bitmap
import android.graphics.Canvas
import android.graphics.Color
import android.graphics.Paint
import android.graphics.drawable.GradientDrawable
import android.content.res.ColorStateList
import android.graphics.Typeface
import android.media.projection.MediaProjectionManager
import android.net.ConnectivityManager
import android.net.Network
import android.net.NetworkCapabilities
import android.net.NetworkRequest
import android.net.Uri
import android.os.Bundle
import android.os.Handler
import android.os.Looper
import android.provider.OpenableColumns
import android.text.TextUtils
import android.text.InputFilter
import android.text.InputType
import android.util.AttributeSet
import android.util.Base64
import android.util.Log
import android.view.Gravity
import android.view.KeyEvent
import android.view.MotionEvent
import android.view.OrientationEventListener
import android.view.ScaleGestureDetector
import android.view.Surface
import android.view.View
import android.view.ViewGroup
import android.view.ViewConfiguration
import android.view.WindowManager
import android.view.inputmethod.EditorInfo
import android.widget.EditText
import android.widget.FrameLayout
import android.widget.ImageView
```

```
import android.widget.LinearLayout
import android.widget.ScrollView
import android.widget.SeekBar
import android.widget.TextView
import android.widget.Toast
import androidx.activity.OnBackPressedCallback
import androidx.activity.result.contract.ActivityResultContracts
import androidx.camera.core.CameraSelector
import androidx.camera.core.ImageCapture
import androidx.camera.core.ImageCaptureException
import androidx.camera.core.Preview
import androidx.camera.lifecycle.ProcessCameraProvider
import androidx.camera.view.PreviewView
import androidx.appcompat.app.AlertDialog
import androidx.appcompat.app.AppCompatActivity
import androidx.core.content.ContextCompat
import androidx.core.content.FileProvider
import androidx.core.view.ViewCompat
import androidx.core.view.WindowInsetsCompat
import androidx.core.view.WindowInsetsControllerCompat
import androidx.core.view.doOnLayout
import com.google.android.material.button.MaterialButton
import com.google.android.material.textfield.TextInputEditText
import com.google.android.material.textfield.TextInputLayout
import okhttp3.MediaType.Companion.toMediaTypeOrNull
import okhttp3.MultipartBody
import okhttp3.OkHttpClient
import okhttp3.Request
import okhttp3.RequestBody
import okhttp3.RequestBody.Companion.toRequestBody
import okio.BufferedSink
import org.json.JSONArray
import org.json.JSONObject
import org.webrtc.SurfaceViewRenderer
import java.io.File
import java.io.IOException
import java.util.concurrent.TimeUnit
import java.util.regex.Pattern

class MainActivity : AppCompatActivity(), SignalingClient.Callback {
    private enum class Screen {
        Auth,
        Connect,
        CoursewareManagement,
        Menu,
        Camera,
        ScreenShare,
        Courseware
    }
```

```
    private enum class CoursewareSubScreen {
        None,
        Source,
        MediaSource,
        ListLoading,
        ServerList,
        ServerMediaList,
        MediaQueue,
        Playback
    }

    private enum class PresentationMode {
        Camera,
        ScreenShare
    }

    private data class CoursewareUploadResult(
        val title: String,
        val url: String
    )

    private data class CoursewareState(
        val title: String,
        val url: String,
        val page: Int,
        val pageCount: Int,
        val screen: Int,
        val screenCount: Int
    )

    private data class StoredCoursewareItem(
        val id: String,
        val title: String,
        val url: String,
        val size: Long,
        val originalUrl: String = "",
        val linkUrl: String = "",
        val lastPage: Int = 1,
        val savedPageCount: Int = 0
    )

    private var currentScreen = Screen.Connect
    @Volatile private var authToken: String? = null
    private var authGeneration = 0
    private var authUsername: String? = null
    private var signalingClient: SignalingClient? = null
    private var webRtcClient: CameraWebRtcClient? = null
    private var cameraRenderer: SurfaceViewRenderer? = null
    private var statusText: TextView? = null
    private var viewerOnline = true
```

```
    private var viewerConnectionNotice: TextView? = null
    private var startLiveButton: MaterialButton? = null
    private var stopLiveButton: MaterialButton? = null
    private var switchCameraButton: MaterialButton? = null
    private var frameLockButton: MaterialButton? = null
    private var torchButton: MaterialButton? = null
    private var rollStudentCameraButton: MaterialButton? = null
    private var pendingStudentRollId: String? = null
    private data class TeachingClass(val id: String, val name: String, val size: Int)
    private var teachingClasses = emptyList<TeachingClass>()
    private var selectedTeachingClassId = ""
    private var studentDrawMode = "name"
    private var studentDrawCount = 50
    private var studentSelectionConfirmed = false
    private var pendingSelectionId: String? = null
    private var classPickerDialog: AlertDialog? = null
    private var setClassPickerSaving: ((Boolean) -> Unit)? = null
    private var classPickerDraft: Triple<String, String, Int>? = null
    private var classPickerLoading = false
    private var classPickerRoom: String? = null
    private var classSettingsButton: MaterialButton? = null
    private var networkGateShowing = false
    private var campusCheckRunning = false
    @Volatile private var selectedServerBaseUrl = BuildConfig.SERVER_BASE_URL
    private fun serverBaseUrl(): String = selectedServerBaseUrl
    private var campusCheckGeneration = 0
    private var audioToggleButton: MaterialButton? = null
    private var cameraControls: LinearLayout? = null
    private var cameraVersionLabel: TextView? = null
    private var orientationListener: OrientationEventListener? = null
    private var rawDeviceRotationDegrees = 0
    private var isUsingFrontCamera = false
    private var activePresentationMode = PresentationMode.Camera
    private var currentDeviceOrientation = createDeviceOrientationPayload(0)
    private var lastSentDeviceOrientation: DeviceOrientationPayload? = null
    private var activeRoomCode: String? = null
    private var roomJoined = false
        set(value) {
            if (field == value) return
            field = value
            applyOrientationPreference()
        }
    private var cameraPausedForBackground = false
    private var restartLiveOnResume = false
    private var resumeCameraAfterJoin = false
    private var resumeLiveAfterJoin = false
    private var pendingCoursewareCloseAfterJoin = false
    private val coursewareMappings = mutableMapOf<String, CoursewarePages>()
    private fun coursewarePages() = coursewareMappings[coursewareUrl]
    private fun displayedCoursewarePage() = coursewarePages()?.slide(coursewarePage) ?:
```

```
        coursewarePage
    private fun displayedCoursewareCount() = coursewarePages()?.slideCount ?:
        coursewarePageCount.coerceAtLeast(1)
    private fun coursewareStateForSlide(slide: Int) = coursewarePages()?.firstState(slide) ?: slide
    private var coursewarePage = 1
    private var coursewarePageCount = 1
    private var coursewareScreen = 1
    private var coursewareScreenCount = 1
    private var coursewareTitle = ""
    private var coursewareUrl = ""
    private var coursewareRememberKey = ""
    /** 大屏浏览器课件容器实际宽高比，未上报时使用 16:9 兼容默认值。 */
    @Volatile private var coursewareViewportAspect = 9f / 16f
    private var coursewareViewportOverlay: CoursewareViewportOverlay? = null
    private var viewerPageRect: android.graphics.RectF? = null
    private var viewerRectPage = 0
    private var viewerRectUrl = ""
    /** 刷新课件列表（如删除后）时保留用户当前的滚动位置。 */
    private var coursewareListScrollY = 0
    private var displayedImportRevision = -1L
    private var importListRefreshSerial = 0
    private var managementPreview: CoursewarePreviewDialog? = null
    /** 本次投屏课件的本地原文件：图片投屏时用它预览，避免从服务器重复下载 */
    private var coursewareLocalUri: Uri? = null
    private var castImageBitmap: Bitmap? = null
    private var videoPlaying = false
    private var videoPosition = 0.0
    private var videoDuration = 0.0
    private var videoUserScrubbing = false
    private var videoPlayPauseButton: MaterialButton? = null
    private var videoSeekBar: SeekBar? = null
    private var videoTimeText: TextView? = null
    private var videoCastStatusText: TextView? = null
    /** 大屏视频的静音/音量（0~100），随大屏端状态上报更新 */
    private var videoMuted = false
    private var videoVolumePercent = 100
    private var videoUserVolumeScrubbing = false
    private var videoVolumeSeekBar: SeekBar? = null
    private var videoMuteButton: MaterialButton? = null
    private var zoomableImageView: ZoomableImageView? = null
    /** 图片投屏当前是否为画笔模式（横竖屏重建后据此恢复） */
    private var imageCastPenMode = false
    /** 画笔模式下的当前颜色与是否板擦，跟随 ZoomableImageView 但需在重建后恢复 */
    private var imageCastPenColor = AnnotationPalette.DEFAULT_COLOR
    private var imageCastPenEraser = false

    // ---- 课件（PDF）本地渲染投屏 ----
    private var coursewarePdfRenderer: CoursewarePdfRenderer? = null
    private var coursewarePdf: CoursewarePdfRenderer.OpenedPdf? = null
    /** 课件渲染失败时降级为遥控页，避免"下载失败 → 重建 → 再失败"形成死循环 */
```

```
    private var coursewarePdfFailed = false
    private var coursewarePdfFailedUrl = ""
    private var coursewarePdfFailureMessage = ""
    /** 课件播放页的翻页动作，由界面注册，供音量键等外部入口复用 */
    private var coursewarePdfPaging: ((delta: Int) -> Unit)? = null
    /** 已渲染的页位图缓存（页码 → 位图），只保留当前页与相邻页，避免内存堆积 */
    private val coursewarePdfBitmaps = LinkedHashMap<Int, Bitmap>()
    /** 按页保存的笔迹：翻页不丢，"撤销 / 清空"只作用于当前页 */
    private val coursewarePdfStrokes = LinkedHashMap<Int, List<AnnotationStroke>>()
    private val coursewareClearHistory = LinkedHashMap<Int, MutableList<List<AnnotationStroke>>>()
    private var annotationsFile = ""
    private var renderedAnnotationsFile = ""
    private var renderedAnnotationsPage = 0
    private var restoringAnnotations = false
    private val embeddedVideos by lazy {
        EmbeddedVideoController(this, coursewareHttpClient,
            getContext = { coursewareUrl.takeIf { mediaKindOf(it) == CastMediaKind.Pdf }?.let {
                EmbeddedVideoController.Context(it, coursewarePage, displayedCoursewarePage()) } },
            absoluteUrl = ::absoluteCoursewareUrl,
            send = { action, url, index -> signalingClient?.sendEmbeddedVideo(action, url, index)
                == true },
            onPages = { url, pages -> coursewareMappings[url] = pages })
    }
    /** 当前已打开课件的缓存键，用于识别切换课件并清理上一份课件的笔迹与位图 */
    private var coursewarePdfOpenedKey = ""
    /** 最近一次收到大屏视频状态的时间，用于判断大屏端是否仍在播放 */
    private var lastVideoStateAtMs = 0L
    /** 最近一次提示“课件控制已重新连接”的时间，避免弱网反复重连时反复弹提示 */
    private var lastCoursewareReconnectToastAtMs = 0L
    /** 最近一次展示“服务器返回错误”提示的时间（节流） */
    private var lastServerErrorShownAtMs = 0L
    /** 重连提示的最小间隔 */
    private val reconnectToastIntervalMs = 15_000L
    /** “服务器返回错误”提示的最小间隔 */
    private val serverErrorToastIntervalMs = 5_000L
    /** 大屏视频状态的有效期：超过该时间没收到状态，就认为大屏端已停止播放 */
    private val videoStateFreshIntervalMs = 60_000L
    private var coursewareUploadInProgress = false
    private var coursewareFastSeekDirection = 0
    private var coursewareFastSeekTargetPage = 1
    private var coursewareFastSeekConsumedClick = false
    private var coursewareFastSeekTicks = 0
    private var coursewareFastSeekRunnable: Runnable? = null
    private var signalReconnectInProgress = false
    private var savedCoursewareState: CoursewareState? = null
    /** 本机图片/视频队列里单个文件的上传状态 */
    private enum class MediaQueueState { Pending, Uploading, Ready, Failed }

    private class LocalMediaItem(
        val uri: Uri,
```

```
        val name: String,
        var state: MediaQueueState = MediaQueueState.Pending,
        var remoteUrl: String = "",
        var remoteTitle: String = ""
    )

    /** 本机图片视频投屏队列：一次可多选，投屏过程中直接切换，不必重新选择文件 */
    private val mediaQueue = mutableListOf<LocalMediaItem>()
    private var mediaQueueIndex = -1
    /** 后台正在依次上传队列中剩余文件（避免重复起线程） */
    private var mediaQueuePreloading = false
    /** 断线重连期间用户已选好文件：连上教室端后自动投屏该序号，不必再选一次 */
    private var pendingMediaQueueIndex: Int? = null
    private var appInForeground = false
    private var lastCoursewareVolumeKeyAtMs = 0L
    private val volumeKeyRepeatIntervalMs = 180L
    // -- 连接/重连状态管理（集中管理，避免状态散落导致不一致）--
    private val reconnectHandler = Handler(Looper.getMainLooper())
    private var reconnectAttempt = 0
    private val maxReconnectAttempts = 5
    private val reconnectTimeoutMs = 15_000L
    private val reconnectBaseDelayMs = 2_000L
    private val reconnectMaxDelayMs = 30_000L
    private var reconnectTimeoutRunnable: Runnable? = null
    private var reconnectRetryRunnable: Runnable? = null
    private var networkCallbackRegistered = false
    private val networkCallback = object : ConnectivityManager.NetworkCallback() {
        override fun onLost(network: Network) {
            runOnUiThread {
                if (appInForeground && currentScreen == Screen.Auth && !campusCheckRunning)
                    checkCampusAccess()
            }
        }
        override fun onAvailable(network: Network) {
            if (appInForeground) switchToCampusNetworkSilently()
            runOnUiThread {
                if (appInForeground && networkGateShowing) checkCampusAccess()
                if (appInForeground) triggerPendingUploadIfReady()
                // 网络恢复且有可用连接码但未连接时，立即重连
                if (appInForeground && activeRoomCode != null &&
                    !roomJoined && !signalReconnectInProgress
                ) {
                    updateStatus("网络已恢复，正在重新连接教室端...")
                    reconnectSignalingForCurrentRoom()
                }
            }
        }
    }

    /** Wi- Fi 恢复后优先探测校园入口；公网使用中也会自动切回内网。 */
```

```
    private fun switchToCampusNetworkSilently() {
        Thread {
            val available = runCatching {
                val request = Request.Builder().url("${BuildConfig.SERVER_BASE_URL.trimEnd('/')}
                    /health").build()
                campusHttpClient.newCall(request).execute().use { response ->
                    if (response.isSuccessful && JSONObject(response.body?.string().orEmpty())
                        .optString("service") == "myclass") true else false
                }
            }.getOrDefault(false)
            if (!available || selectedServerBaseUrl == BuildConfig.SERVER_BASE_URL || isDestroyed)
                return@Thread
            runOnUiThread {
                if (isDestroyed) return@runOnUiThread
                selectedServerBaseUrl = BuildConfig.SERVER_BASE_URL
                CoursewareApi.serverBaseUrl = selectedServerBaseUrl
                if (activeRoomCode != null && !signalReconnectInProgress) {
                    reconnectSignalingForCurrentRoom()
                }
            }
        }.start()
    }
    private var coursewareSubScreen: CoursewareSubScreen = CoursewareSubScreen.None
    private var coursewareReturnList: CoursewareSubScreen? = null
    private var clipboardCheckRunning = false
    private var clipboardManualRequested = false
    private var clipboardDialog: AlertDialog? = null
    private val clipboardPromptPrefs: SharedPreferences
        get() = getSharedPreferences("myclass_clipboard", Context.MODE_PRIVATE)
    private var initialIntentProcessed = false
    private val coursewareHttpClient = OkHttpClient.Builder()
        .addInterceptor { chain ->
            val response = chain.proceed(chain.request())
            if (response.code == 401) {
                val token = chain.request().header("Authorization")?.removePrefix("Bearer ")
                onAuthenticationExpired(token)
            }
            response
        }
        .connectTimeout(30, TimeUnit.SECONDS)
        .writeTimeout(30, TimeUnit.MINUTES)
        .readTimeout(30, TimeUnit.MINUTES)
        .callTimeout(35, TimeUnit.MINUTES)
        .retryOnConnectionFailure(true)
        .build()

    private val authHttpClient = OkHttpClient.Builder()
        .connectTimeout(4, TimeUnit.SECONDS)
        .readTimeout(4, TimeUnit.SECONDS)
        .callTimeout(6, TimeUnit.SECONDS)
```

```
        .build()

    private val campusHttpClient = OkHttpClient.Builder()
        .connectTimeout(3, TimeUnit.SECONDS)
        .readTimeout(3, TimeUnit.SECONDS)
        .callTimeout(4, TimeUnit.SECONDS)
        .retryOnConnectionFailure(false)
        .build()

    private val prefs: SharedPreferences
        get() = getSharedPreferences("myclass_auth", Context.MODE_PRIVATE)

    private val permissionLauncher = registerForActivityResult(
        ActivityResultContracts.RequestMultiplePermissions()
    ) { result ->
        if (result.values.all { it }) {
            showCameraScreen()
        } else {
            toast("需要摄像头和麦克风权限才能直播")
        }
    }

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        applyOrientationPreference()
        rawDeviceRotationDegrees = displayRotationDegrees()
        currentDeviceOrientation = createDeviceOrientationPayload(rawDeviceRotationDegrees)
        window.addFlags(WindowManager.LayoutParams.FLAG_KEEP_SCREEN_ON)
        onBackPressedDispatcher.addCallback(this, backCallback)
        registerNetworkCallback()
        loadAuth()
        ExternalFileReceiver.restoreQueue(this)
        updateManager = UpdateManager(this).also { it.checkForUpdate() }
        checkCampusAccess()
        initialIntentProcessed = savedInstanceState?.getBoolean("initialIntentProcessed") ?: false
        if (!initialIntentProcessed) {
            initialIntentProcessed = true
            val capturedIntent = intent
            handleShareIntent(capturedIntent)
        }
    }

    override fun onSaveInstanceState(outState: Bundle) {
        outState.putBoolean("initialIntentProcessed", initialIntentProcessed)
        super.onSaveInstanceState(outState)
    }

    override fun onNewIntent(intent: Intent) {
        super.onNewIntent(intent)
        setIntent(intent)
```

```
        handleShareIntent(intent)
    }

    private fun handleShareIntent(intent: Intent?) {
        if (intent?.action !in listOf(Intent.ACTION_SEND, Intent.ACTION_VIEW)) return
        val targetIntent = intent ?: return
        val normalizedIntent = normalizeLinkIntent(targetIntent)
        ImportUploadService.receive(this, normalizedIntent)
    }

    override fun onConfigurationChanged(newConfig: Configuration) {
        super.onConfigurationChanged(newConfig)
        val reopenClassPicker = classPickerDialog?.isShowing == true
        if (reopenClassPicker) classPickerDialog?.dismiss()
        // 横竖屏切换时重建当前页面布局
        when (currentScreen) {
            Screen.Auth -> if (networkGateShowing) showCampusNetworkScreen(
                if (campusCheckRunning) "请稍候，正在检测校园投屏服务…" else campusNetworkMessage(),
                    campusCheckRunning
            ) else showAuthScreen()
            Screen.Connect -> showConnectScreen()
            Screen.CoursewareManagement -> showCoursewareManagement()
            Screen.Menu -> showMenuScreen()
            Screen.Camera -> {
                // 不重建整个相机页面（会中断预览），仅更新按钮布局和悬浮按钮位置
                updateCameraControlsLayout()
                updateAudioButtonPosition()
                cameraRenderer?.requestLayout()
                refreshLockedFramePreviewAfterLayout()
            }
            Screen.ScreenShare -> {
                // 共享屏幕页面布局简单，无需重建；但虚拟显示必须跟着屏幕方向 resize。
                webRtcClient?.refreshScreenCaptureSize()
            }
            Screen.Courseware -> {
                // 根据子页面类型重建课件界面；正在拍照时重建拍照页本身，
                // 否则横竖屏一转就把老师踢回来源页
                if (captureUiRoot != null) {
                    showCaptureScreen()
                } else when (coursewareSubScreen) {
                    CoursewareSubScreen.Source -> showCoursewareSourceScreen()
                    CoursewareSubScreen.MediaSource -> showMediaCastSourceScreen()
                    CoursewareSubScreen.ListLoading -> showCoursewareListLoadingScreen()
                    CoursewareSubScreen.ServerList -> {
                        // 服务器列表由异步数据填充，此处重建为加载中状态
                        showCoursewareListLoadingScreen()
                        loadServerCoursewareList()
                    }
                    CoursewareSubScreen.ServerMediaList -> loadServerMediaList()
                    CoursewareSubScreen.MediaQueue -> showMediaQueueScreen()
```

```
                CoursewareSubScreen.Playback -> showCoursewareScreen(
                    title = coursewareTitle,
                    isUploading = coursewareUploadInProgress
                )
                CoursewareSubScreen.None -> {}
            }
        }
    }
    if (reopenClassPicker) showPhoneClassPicker()
    syncDeviceRotationFromDisplay(force = true)
}

override fun onResume() {
    super.onResume()
    appInForeground = true
    window.decorView.post { checkClipboardLink() }
    ImportUploadService.observe(this)
    refreshListAfterImport()
    triggerPendingUploadIfReady()
    // 从「允许安装未知应用」设置页返回：已授权就复用已下载的安装包继续安装，不重新下载
    updateManager?.onResumeFromSettings()
    if (networkGateShowing) checkCampusAccess()
    if (currentScreen == Screen.Camera && cameraPausedForBackground) {
        val shouldRestartLive = restartLiveOnResume
        val roomCode = activeRoomCode
        cameraPausedForBackground = false
        restartLiveOnResume = false
        if (roomCode == null) {
            toast("请重新输入连接码")
            showConnectScreen()
            return
        }
        showCameraScreen()
        updateStatus("正在重新连接教室端...")
        connectToRoom(
            code = roomCode,
            resumeCameraAfterJoin = true,
            resumeLiveAfterJoin = shouldRestartLive
        )
    } else if (activeRoomCode != null && !roomJoined && !signalReconnectInProgress) {
        // 其它页面从后台返回时的连接健康检查：
        // 后台期间信令可能已断开但 UI 仍显示已连接，这里主动补连
        updateStatus("正在重新连接教室端...")
        reconnectSignalingForCurrentRoom()
    }
}

// 图片/视频投屏：走课件上传链路，上传后由服务器下发 URL 播放（清晰流畅，不经过 WebRTC 编码）
// 支持一次选择多个文件，投屏过程中通过“上一个/下一个/媒体列表”直接切换
private val mediaPickerLauncher = registerForActivityResult(
```

```
        ActivityResultContracts.OpenMultipleDocuments()
    ) { uris: List<Uri>? ->
        val picked = uris.orEmpty().filter { it != Uri.EMPTY }
        if (picked.isEmpty()) return@registerForActivityResult
        // 逐个保留读取权限，上传完成后仍可用原文件做本地预览
        picked.forEach { uri ->
            runCatching {
                contentResolver.takePersistableUriPermission(
                    uri,
                    Intent.FLAG_GRANT_READ_URI_PERMISSION
                )
            }
        }
        startLocalMediaQueue(picked)
    }

    // 应用内拍照投屏：预览常驻、快门可连按，拍完立刻上传投屏；
    // 不走系统相机的“完成 / 重拍”确认，因此可以连续拍多张
    private var captureSessionActive = false
    private var captureShotCount = 0
    private var captureUiRoot: View? = null
    private var capturePreviewView: PreviewView? = null
    private var captureImageCapture: ImageCapture? = null
    private var captureCameraProvider: ProcessCameraProvider? = null
    private var captureThumbRow: LinearLayout? = null
    private var captureThumbScroller: ScrollView? = null
    /** 缩略图是否竖排（横屏右侧栏） */
    private var captureThumbVertical = false
    private var captureCountLabel: TextView? = null
    /** 本次连拍会话已拍下的照片文件，用于重新进入拍照页时复原缩略图 */
    private val captureFiles = mutableListOf<File>()

    private val capturePermissionLauncher = registerForActivityResult(
        ActivityResultContracts.RequestPermission()
    ) { granted ->
        if (granted) showCaptureScreen() else toast("需要相机权限才能拍照投屏")
    }

    private val coursewarePickerLauncher = registerForActivityResult(
        ActivityResultContracts.GetContent()
    ) { uri: Uri? ->
        uri ?: return@registerForActivityResult
        uploadCourseware(uri)
    }

    private val screenCaptureLauncher = registerForActivityResult(
        ActivityResultContracts.StartActivityForResult()
    ) { result ->
        val data = result.data
        if (result.resultCode == Activity.RESULT_OK && data != null) {
```

```
            startScreenShare(data)
        } else {
            toast("已取消屏幕共享")
        }
    }

    override fun onPause() {
        appInForeground = false
        if (currentScreen == Screen.Camera && !isFinishing) {
            cameraPausedForBackground = true
            restartLiveOnResume = webRtcClient?.isLive() == true
            if (restartLiveOnResume) {
                signalingClient?.sendStop()
            }
            releaseCamera()
            roomJoined = false
            signalingClient?.close()
            signalingClient = null
        }
        super.onPause()
    }

    override fun onDestroy() {
        managementPreview?.dismiss()
        embeddedVideos.destroy()
        ImportUploadService.detach(this)
        clipboardDialog?.dismiss()
        campusCheckGeneration += 1
        classPickerDialog?.dismiss()
        unregisterNetworkCallback()
        cancelReconnectTimeout()
        cancelReconnectRetry()
        orientationListener?.disable()
        releaseCamera()
        signalingClient?.close()
        super.onDestroy()
    }

    private fun registerNetworkCallback() {
        val connectivityManager =
            getSystemService(Context.CONNECTIVITY_SERVICE) as? ConnectivityManager ?: return
        val request = NetworkRequest.Builder()
            .addCapability(NetworkCapabilities.NET_CAPABILITY_INTERNET)
            .build()
        runCatching {
            connectivityManager.registerNetworkCallback(request, networkCallback)
            networkCallbackRegistered = true
        }
    }
```

```
    private fun unregisterNetworkCallback() {
        if (!networkCallbackRegistered) return
        val connectivityManager =
            getSystemService(Context.CONNECTIVITY_SERVICE) as? ConnectivityManager ?: return
        runCatching {
            connectivityManager.unregisterNetworkCallback(networkCallback)
            networkCallbackRegistered = false
        }
    }

    private val backCallback = object : OnBackPressedCallback(true) {
        override fun handleOnBackPressed() {
            if (quickPagePopup?.isShowing == true) {
                quickPagePopup?.dismiss()
                return
            }
            when (currentScreen) {
                Screen.Auth -> {
                    isEnabled = false
                    onBackPressedDispatcher.onBackPressed()
                }
                Screen.Camera -> {
                    returnToMenuFromCamera()
                }
                Screen.ScreenShare -> {
                    stopScreenShareAndReturnMenu()
                }
                Screen.Courseware -> {
                    closeCoursewareAndReturnMenu()
                }
                Screen.Menu -> {
                    signalingClient?.close()
                    signalingClient = null
                    activeRoomCode = null
                    viewerPageRect = null
                    coursewareViewportAspect = 9f / 16f
                    classPickerRoom = null
                    studentSelectionConfirmed = false
                    roomJoined = false
                    showConnectScreen()
                }
                Screen.Connect -> {
                    isEnabled = false
                    onBackPressedDispatcher.onBackPressed()
                }
                Screen.CoursewareManagement -> showConnectScreen()
            }
        }
    }
```

```
    // -- 认证相关 --
    private fun loadAuth() {
        val p = prefs
        authToken = p.getString("token", null)
        authUsername = p.getString("username", null)
    }

    private fun saveAuth(token: String, username: String) {
        authGeneration += 1
        authToken = token
        authUsername = username
        prefs.edit()
            .putString("token", token)
            .putString("username", username)
            .apply()
    }

    private fun clearAuth() {
        managementPreview?.dismiss()
        clipboardDialog?.dismiss()
        authGeneration += 1
        authToken = null
        authUsername = null
        teachingClasses = emptyList()
        selectedTeachingClassId = ""
        studentSelectionConfirmed = false
        classPickerRoom = null
        classPickerDraft = null
        prefs.edit().remove("token").remove("username").apply()
    }

    private fun triggerPendingUploadIfReady() {
        ImportUploadService.start(this)
    }

    fun onImportStatus(message: String, finished: Boolean, rejectedToken: String?) {
        if (isDestroyed || !appInForeground) return
        if (currentScreen == Screen.Connect || currentScreen == Screen.Menu) updateStatus(message)
        if (finished) toast(message)
        if (finished && rejectedToken == null) refreshListAfterImport()
        if (rejectedToken != null) onAuthenticationExpired(rejectedToken)
    }

    private fun importRevision(): Long = getSharedPreferences("myclass_import_updates",
        MODE_PRIVATE)
        .getLong(authUsername.orEmpty(), 0L)

    private fun refreshListAfterImport() {
        val management = currentScreen == Screen.CoursewareManagement
        val media = currentScreen == Screen.Courseware && coursewareSubScreen ==
```

```
            CoursewareSubScreen.ServerMediaList
        val server = currentScreen == Screen.Courseware && coursewareSubScreen ==
            CoursewareSubScreen.ServerList
        val revision = importRevision()
        if ((!management && !media && !server) || revision == displayedImportRevision || authToken
            == null) return
        val screen = currentScreen
        val subScreen = coursewareSubScreen
        val generation = authGeneration
        val serial = ++importListRefreshSerial
        val content = findViewById<ViewGroup>(android.R.id.content)
        val originalRoot = content.getChildAt(0)
        fun findScroll(view: View): ScrollView? {
            if (view is ScrollView) return view
            if (view is ViewGroup) for (i in 0 until view.childCount) {
                findScroll(view.getChildAt(i))?.let { return it }
            }
            return null
        }
        originalRoot?.let { findScroll(it)?.let { scroll -> coursewareListScrollY = scroll.scrollY }
            }
        Thread {
            runCatching { fetchServerCoursewareListBlocking() }.onSuccess { items ->
                runOnUiThread {
                    if (isDestroyed || serial != importListRefreshSerial || generation !=
                        authGeneration ||
                        currentScreen != screen || coursewareSubScreen != subScreen ||
                            content.getChildAt(0) !== originalRoot) return@runOnUiThread
                    when {
                        management -> showCoursewareManagementList(items)
                        media -> showServerMediaList(items.filter {
                            it.linkUrl.isNotBlank() || mediaKindOf(it.url) in
                                listOf(CastMediaKind.Image, CastMediaKind.Video)
                        })
                        else -> showServerCoursewareList(items)
                    }
                    displayedImportRevision = revision
                    refreshListAfterImport()
                }
            }
        }.start()
    }

    private fun addAuthHeader(builder: Request.Builder) {
        authToken?.let { builder.header("Authorization", "Bearer $it") }
    }

    private fun verifyTokenThenProceed() {
        val token = authToken ?: return
        val generation = authGeneration
```

```
        Thread {
            runCatching {
                val request = Request.Builder()
                    .url("${serverBaseUrl().trimEnd('/')}/api/auth/me")
                    .get().header("Authorization", "Bearer $token")
                    .build()
                authHttpClient.newCall(request).execute().use { response ->
                    if (response.code == 401) throw TokenExpiredException()
                    if (!response.isSuccessful) throw IOException("HTTP ${response.code}")
                }
            }.onSuccess {
                runOnUiThread {
                    if (authGeneration == generation && currentScreen == Screen.Auth) {
                        showConnectScreen()
                        triggerPendingUploadIfReady()
                    }
                }
            }.onFailure { error ->
                runOnUiThread {
                    // A slow startup request must not clear a newer login.
                    if (authGeneration != generation || currentScreen != Screen.Auth)
                        return@runOnUiThread
                    if (error is TokenExpiredException) {
                        clearAuth()
                        updateStatus("登录已过期，请重新登录")
                    } else {
                        showCampusNetworkScreen(campusNetworkMessage())
                    }
                }
            }
        }.start()
    }

    private class TokenExpiredException : IOException()
    private class AuthResponseException(message: String) : IOException(message)

    private fun campusNetworkMessage(): String {
        val manager = getSystemService(Context.CONNECTIVITY_SERVICE) as? ConnectivityManager
        val network = manager?.activeNetwork
        val capabilities = network?.let { manager.getNetworkCapabilities(it) }
        return when {
            network == null -> "当前没有网络。请连接 Wi-Fi 或移动网络后重试。"
            capabilities?.hasTransport(NetworkCapabilities.TRANSPORT_WIFI) != true ->
                "校园网不可用，可使用公网地址继续；建议连接 Wi-Fi 以获得更稳定的投屏体验。"
            else -> "校园网地址暂不可达，可使用公网地址继续；建议连接 Wi-Fi
                以获得更稳定的投屏体验。"
        }
    }

    private fun showCampusNetworkScreen(message: String, checking: Boolean = false) {
```

```
        if (isDestroyed || isFinishing) return
        currentScreen = Screen.Auth
        networkGateShowing = true
        val root = baseColumn().apply { setPadding(dp(28), dp(32), dp(28), dp(32)) }
        root.addView(titleText(if (checking) "正在连接服务器" else "连接服务器", 24f))
        statusText = bodyText(message).apply { gravity = Gravity.CENTER }
        root.addView(statusText)
        if (!checking) root.addView(bodyText("公网入口：https://sz.imst.xyz/myclass/\n建议连接
            Wi-Fi 使用课堂投屏").apply { gravity = Gravity.CENTER })
        root.addView(primaryButton("打开 Wi-Fi 设置").apply {
            setOnClickListener { startActivity(Intent(android.provider.Settings.ACTION_WIFI_SETTINGS
                )) }
        })
        root.addView(secondaryButton(if (checking) "正在检测…" else "重新检测").apply {
            isEnabled = !checking
            setOnClickListener { checkCampusAccess() }
        })
        root.addView(versionLabel())
        showScreenView(root)
    }

    private fun checkCampusAccess() {
        if (campusCheckRunning || isDestroyed) return
        campusCheckRunning = true
        val generation = ++campusCheckGeneration
        showCampusNetworkScreen("请稍候，正在检测校园投屏服务…", checking = true)
        Thread {
            val available = listOf(BuildConfig.SERVER_BASE_URL,
                BuildConfig.EXTERNAL_SERVER_BASE_URL)
                .firstNotNullOfOrNull { base ->
                    runCatching {
                        val request = Request.Builder().url("${base.trimEnd('/')}/health").build()
                        campusHttpClient.newCall(request).execute().use { response ->
                            if (response.isSuccessful && JSONObject(response.body?.string()
                                .orEmpty()).optString("service") == "myclass") base else null
                        }
                    }.getOrNull()
                }
            runOnUiThread {
                if (isDestroyed || generation != campusCheckGeneration) return@runOnUiThread
                campusCheckRunning = false
                if (available != null) {
                    selectedServerBaseUrl = available
                    CoursewareApi.serverBaseUrl = available
                    networkGateShowing = false
                    showAuthScreen()
                    if (authToken != null) {
                        updateStatus(if (available == BuildConfig.SERVER_BASE_URL)
                            "校园网络已连接，正在恢复登录…" else "已通过公网连接，建议连接 Wi-Fi 使用投屏")
                        verifyTokenThenProceed()
```

```
                }
            } else showCampusNetworkScreen(campusNetworkMessage())
        }
    }.start()
}

private val authRecheckRunning = java.util.concurrent.atomic.AtomicBoolean(false)

fun onAuthenticationExpired(rejectedToken: String?) {
    if (rejectedToken == null || rejectedToken != authToken ||
        !authRecheckRunning.compareAndSet(false, true)) return
    Thread {
        try {
            val request = Request.Builder().url("${serverBaseUrl().trimEnd('/')}/api/auth/me")
                .header("Authorization", "Bearer $rejectedToken").build()
            val expired = authHttpClient.newCall(request).execute().use { it.code == 401 }
            if (expired) showAuthenticationExpired(rejectedToken)
        } catch (e: IOException) {
            Log.w("MyClassAuth", "Session verification unavailable; keeping credentials", e)
        } finally {
            authRecheckRunning.set(false)
        }
    }.start()
}

private fun showAuthenticationExpired(rejectedToken: String) {
    runOnUiThread {
        if (authToken != rejectedToken || prefs.getString("token", null) != rejectedToken ||
            isDestroyed) return@runOnUiThread
        clearAuth()
        // Preserve the active projection until the teacher chooses to sign in.
        androidx.appcompat.app.AlertDialog.Builder(this)
            .setTitle("登录状态已失效")
            .setMessage("投屏连接与账号登录是独立状态。当前投屏可继续；使用课件列表或上传需要重
                新登录。")
            .setNegativeButton("稍后登录", null)
            .setPositiveButton("重新登录") { _, _ ->
                signalingClient?.sendStop()
                signalingClient?.sendCoursewareClose()
                releaseCamera()
                handleRoomInvalid()
                showAuthScreen()
                updateStatus("请重新登录后连接课堂")
            }
            .show()
    }
}

private fun showAuthScreen() {
    networkGateShowing = false
```

```
        currentScreen = Screen.Auth
        val isLandscape = resources.configuration.orientation == Configuration.ORIENTATION_LANDSCAPE

        // 创建输入控件
        val usernameLayout = TextInputLayout(this).apply {
            hint = "用户名（2-20 位）"
            boxBackgroundMode = TextInputLayout.BOX_BACKGROUND_OUTLINE
            layoutParams = LinearLayout.LayoutParams(
                ViewGroup.LayoutParams.MATCH_PARENT,
                ViewGroup.LayoutParams.WRAP_CONTENT
            ).apply { topMargin = dp(if (isLandscape) 0 else 28) }
        }
        val usernameInput = TextInputEditText(usernameLayout.context).apply {
            inputType = InputType.TYPE_CLASS_TEXT
            textSize = 17f
            setSingleLine(true)
        }
        usernameLayout.addView(usernameInput)

        val passwordLayout = TextInputLayout(this).apply {
            hint = "密码（4-32 位）"
            boxBackgroundMode = TextInputLayout.BOX_BACKGROUND_OUTLINE
            layoutParams = LinearLayout.LayoutParams(
                ViewGroup.LayoutParams.MATCH_PARENT,
                ViewGroup.LayoutParams.WRAP_CONTENT
            ).apply { topMargin = dp(16) }
        }
        val passwordInput = TextInputEditText(passwordLayout.context).apply {
            inputType = InputType.TYPE_CLASS_TEXT or InputType.TYPE_TEXT_VARIATION_PASSWORD
            textSize = 17f
            setSingleLine(true)
        }
        passwordLayout.addView(passwordInput)

        val loginButton = primaryButton("登录").apply {
            layoutParams = LinearLayout.LayoutParams(
                ViewGroup.LayoutParams.MATCH_PARENT,
                dp(52)
            ).apply { topMargin = dp(24) }
            setOnClickListener {
                val name = usernameInput.text?.toString()?.trim().orEmpty()
                val pass = passwordInput.text?.toString().orEmpty()
                if (name.length < 2) { usernameLayout.error = "用户名至少 2 位";
                    return@setOnClickListener }
                if (pass.length < 4) { passwordLayout.error = "密码至少 4 位";
                    return@setOnClickListener }
                usernameLayout.error = null
                passwordLayout.error = null
                isEnabled = false
                text = "登录中..."
```

```
            performAuth(false, name, pass)
        }
    }
    val registerButton = secondaryButton("注册新账号").apply {
        layoutParams = LinearLayout.LayoutParams(
            ViewGroup.LayoutParams.MATCH_PARENT,
            dp(52)
        ).apply { topMargin = dp(12) }
        setOnClickListener {
            val name = usernameInput.text?.toString()?.trim().orEmpty()
            val pass = passwordInput.text?.toString().orEmpty()
            if (name.length < 2) { usernameLayout.error = "用户名至少 2 位";
                return@setOnClickListener }
            if (!Regex("^[\\u4e00-\\u9fa5a-zA-Z0-9_]+$").matches(name)) {
                usernameLayout.error = "仅支持中英文数字下划线"; return@setOnClickListener
            }
            if (name.length > 20) { usernameLayout.error = "用户名最多 20 位";
                return@setOnClickListener }
            if (pass.length < 4) { passwordLayout.error = "密码至少 4 位";
                return@setOnClickListener }
            if (pass.length > 32) { passwordLayout.error = "密码最多 32 位";
                return@setOnClickListener }
            usernameLayout.error = null
            passwordLayout.error = null
            isEnabled = false
            text = "注册中..."
            performAuth(true, name, pass)
        }
    }
    val statusView = bodyText("").apply {
        gravity = Gravity.CENTER
        layoutParams = LinearLayout.LayoutParams(
            ViewGroup.LayoutParams.MATCH_PARENT,
            ViewGroup.LayoutParams.WRAP_CONTENT
        ).apply { topMargin = dp(18) }
    }

    statusText = statusView
    if (isLandscape) {
        // 横屏：左右结构
        val root = landscapeRoot().apply {
            setPadding(dp(24), dp(16), dp(24), dp(16))
            gravity = Gravity.CENTER
        }
        val leftPanel = baseColumn().apply {
            gravity = Gravity.CENTER
            layoutParams = LinearLayout.LayoutParams(0, ViewGroup.LayoutParams.WRAP_CONTENT, 1f)
                .apply {
                marginEnd = dp(24)
            }
```

```
        }
        leftPanel.addView(titleText("上课投屏平台", 22f))
        leftPanel.addView(bodyText("首次使用请注册账号").apply {
            gravity = Gravity.CENTER
            setTextColor(Color.argb(180, 96, 96, 96))
            layoutParams = LinearLayout.LayoutParams(
                ViewGroup.LayoutParams.MATCH_PARENT,
                ViewGroup.LayoutParams.WRAP_CONTENT
            ).apply { topMargin = dp(8) }
        })
        leftPanel.addView(statusView)
        leftPanel.addView(footerLabel().apply {
            (layoutParams as LinearLayout.LayoutParams).topMargin = dp(16)
        })
        leftPanel.addView(versionLabel())

        val rightPanel = baseColumn().apply {
            gravity = Gravity.CENTER
            layoutParams = LinearLayout.LayoutParams(0, ViewGroup.LayoutParams.WRAP_CONTENT, 1f)
        }
        rightPanel.addView(usernameLayout)
        rightPanel.addView(passwordLayout)
        rightPanel.addView(loginButton)
        rightPanel.addView(registerButton)

        root.addView(leftPanel)
        root.addView(rightPanel)
        showScreenView(root)
    } else {
        // 竖屏：垂直结构
        val root = baseColumn().apply {
            setPadding(dp(28), dp(36), dp(28), dp(32))
        }
        root.addView(titleText("上课投屏平台", 24f))
        root.addView(bodyText("首次使用请注册账号").apply {
            gravity = Gravity.CENTER
            setTextColor(Color.argb(180, 96, 96, 96))
            layoutParams = LinearLayout.LayoutParams(
                ViewGroup.LayoutParams.MATCH_PARENT,
                ViewGroup.LayoutParams.WRAP_CONTENT
            ).apply { topMargin = dp(8) }
        })
        root.addView(usernameLayout)
        root.addView(passwordLayout)
        root.addView(loginButton)
        root.addView(registerButton)
        root.addView(statusView)
        root.addView(footerLabel())
        root.addView(versionLabel())
        showScreenView(root)
```

```
        }
    }

    private fun performAuth(isRegister: Boolean, username: String, password: String) {
        Thread {
            runCatching {
                val json = JSONObject().apply {
                    put("username", username)
                    put("password", password)
                }
                val body = json.toString()
                    .toRequestBody("application/json; charset=utf-8".toMediaTypeOrNull())
                val url = "${serverBaseUrl().trimEnd('/')}/api/auth/${if (isRegister) "register"
                    else "login"}"
                val request = Request.Builder().url(url).post(body).build()
                authHttpClient.newCall(request).execute().use { response ->
                    val respBody = response.body?.string().orEmpty()
                    if (!response.isSuccessful) {
                        val error = runCatching {
                            JSONObject(respBody).optString("error")
                        }.getOrNull().orEmpty()
                        throw AuthResponseException(error.ifBlank { "服务器返回 HTTP
                            ${response.code}" })
                    }
                    val resp = JSONObject(respBody)
                    val token = resp.getString("token")
                    val name = resp.getString("username")
                    runOnUiThread {
                        saveAuth(token, name)
                        triggerPendingUploadIfReady()
                        showConnectScreen()
                        toast("${if (isRegister) "注册" else "登录"}成功，欢迎 $name")
                    }
                }
            }.onFailure { error ->
                runOnUiThread {
                    if (isDestroyed) return@runOnUiThread
                    if (error is AuthResponseException) {
                        showAuthScreen()
                        updateStatus(error.message ?: "认证失败")
                    } else showCampusNetworkScreen(campusNetworkMessage())
                }
            }
        }.start()
    }

    private fun browserEntryAddress(): String =
        serverBaseUrl().removePrefix("https://").removePrefix("http://").trimEnd('/')

    private fun showConnectScreen() {
```

```
        currentScreen = Screen.Connect
        window.decorView.post { checkClipboardLink() }
        val isLandscape = resources.configuration.orientation == Configuration.ORIENTATION_LANDSCAPE

        val inputLayout = TextInputLayout(this).apply {
            hint = "连接码"
            boxBackgroundMode = TextInputLayout.BOX_BACKGROUND_OUTLINE
            layoutParams = LinearLayout.LayoutParams(
                ViewGroup.LayoutParams.MATCH_PARENT,
                ViewGroup.LayoutParams.WRAP_CONTENT
            ).apply {
                topMargin = dp(if (isLandscape) 0 else 28)
            }
        }
        val codeInput = TextInputEditText(inputLayout.context).apply {
            inputType = InputType.TYPE_CLASS_NUMBER
            filters = arrayOf(InputFilter.LengthFilter(4))
            gravity = Gravity.CENTER
            textSize = 28f
            setSingleLine(true)
            val lastCode = prefs.getString("last_connect_code", "").orEmpty()
            if (lastCode.length == 4) {
                setText(lastCode)
                setSelection(lastCode.length)
            }
        }
        inputLayout.addView(codeInput)

        val connectButton = primaryButton("连接").apply {
            layoutParams = LinearLayout.LayoutParams(
                ViewGroup.LayoutParams.MATCH_PARENT,
                dp(56)
            ).apply {
                topMargin = dp(18)
            }
            setOnClickListener {
                val code = codeInput.text?.toString()?.trim().orEmpty()
                if (!Regex("^\\d{4}$").matches(code)) {
                    inputLayout.error = "请输入 4 位数字连接码"
                    return@setOnClickListener
                }
                inputLayout.error = null
                isEnabled = false
                text = "连接中..."
                prefs.edit().putString("last_connect_code", code).apply()
                connectToRoom(code)
            }
        }

        statusText = bodyText("请输入教室大屏上的 4 位连接码").apply {
```

```
            gravity = Gravity.CENTER
            layoutParams = LinearLayout.LayoutParams(
                ViewGroup.LayoutParams.MATCH_PARENT,
                ViewGroup.LayoutParams.WRAP_CONTENT
            ).apply {
                topMargin = dp(20)
            }
        }

        val hintText = TextView(this).apply {
            text = "浏览器进入 ${browserEntryAddress()} 查看连接码"
            textSize = 13f
            gravity = Gravity.CENTER
            alpha = 0.6f
            setTextColor(ContextCompat.getColor(this@MainActivity, R.color.myclass_on_surface))
            layoutParams = LinearLayout.LayoutParams(
                ViewGroup.LayoutParams.MATCH_PARENT,
                ViewGroup.LayoutParams.WRAP_CONTENT
            ).apply {
                topMargin = dp(6)
            }
        }

        val userButton = secondaryButton("用户管理：${authUsername ?: ""}").apply {
            textSize = 14f
            layoutParams = LinearLayout.LayoutParams(
                ViewGroup.LayoutParams.WRAP_CONTENT,
                dp(40)
            ).apply {
                topMargin = dp(32)
            }
            setOnClickListener {
                showUserManagementDialog()
            }
        }

        if (ExternalFileReceiver.pendingCount() > 0) {
            updateStatus("${ExternalFileReceiver.pendingCount()}
                个课件等待上传，登录后自动上传，无需连接教室")
        }

        if (isLandscape) {
            // 横屏：左右结构
            val root = landscapeRoot().apply {
                setPadding(dp(24), dp(16), dp(24), dp(16))
                gravity = Gravity.CENTER
            }
            val leftPanel = baseColumn().apply {
                gravity = Gravity.CENTER
                layoutParams = LinearLayout.LayoutParams(0, ViewGroup.LayoutParams.WRAP_CONTENT, 1f)
```

```
                .apply {
                marginEnd = dp(24)
            }
        }
        leftPanel.addView(titleText(getString(R.string.platform_title), 22f))
        leftPanel.addView(statusText)
        leftPanel.addView(hintText)
        leftPanel.addView(userButton.apply {
            (layoutParams as LinearLayout.LayoutParams).topMargin = dp(16)
        })

        val rightPanel = baseColumn().apply {
            gravity = Gravity.CENTER
            layoutParams = LinearLayout.LayoutParams(0, ViewGroup.LayoutParams.WRAP_CONTENT, 1f)
        }
        rightPanel.addView(inputLayout)
        rightPanel.addView(connectButton)
        rightPanel.addView(versionLabel().apply {
            (layoutParams as LinearLayout.LayoutParams).topMargin = dp(12)
        })
        rightPanel.addView(footerLabel().apply {
            (layoutParams as LinearLayout.LayoutParams).topMargin = dp(2)
        })

        root.addView(leftPanel)
        root.addView(rightPanel)
        showScreenView(root)
    } else {
        // 竖屏：垂直结构
        val root = baseColumn().apply {
            setPadding(dp(28), dp(36), dp(28), dp(20))
        }
        root.addView(titleText(getString(R.string.platform_title), 22f))
        root.addView(inputLayout)
        root.addView(connectButton)
        root.addView(statusText)
        root.addView(hintText)
        root.addView(userButton)
        root.addView(versionLabel().apply {
            (layoutParams as LinearLayout.LayoutParams).topMargin = dp(6)
        })
        root.addView(footerLabel().apply {
            (layoutParams as LinearLayout.LayoutParams).topMargin = dp(2)
        })
        showScreenView(root)
    }
}

private fun showUserManagementDialog() {
    AlertDialog.Builder(this)
```

```
            .setTitle("用户管理")
            .setItems(arrayOf("课件管理", "后台管理", "剪贴板链接", "切换用户")) { _, which ->
                when (which) {
                    0 -> showCoursewareManagement()
                    1 -> openAdminConsole()
                    2 -> {
                        clipboardManualRequested = true
                        window.decorView.post { checkClipboardLink() }
                    }
                    3 -> confirmSwitchUser()
                }
            }
            .setNegativeButton("取消", null)
            .show()
    }

    /** 管理后台是网页端，手机浏览器打开后用它自己的管理员登录。 */
    private fun adminConsoleUrl(): String = serverBaseUrl().trimEnd('/') + "/admin.html"

    private fun openAdminConsole() {
        runCatching { startActivity(Intent(Intent.ACTION_VIEW, Uri.parse(adminConsoleUrl()))) }
            .onSuccess { toast("已在浏览器中打开管理后台") }
            .onFailure { toast("没有可打开管理后台的浏览器") }
    }

    private fun confirmSwitchUser() {
        AlertDialog.Builder(this)
            .setTitle("切换用户")
            .setMessage("确定要退出当前账号（${authUsername}）吗？")
            .setNegativeButton("取消", null)
            .setPositiveButton("确定") { _, _ ->
                signalingClient?.close()
                signalingClient = null
                activeRoomCode = null
                viewerPageRect = null
                coursewareViewportAspect = 9f / 16f
                roomJoined = false
                // Pending imports retain their owner and resume only for that account.
                clearAuth()
                showAuthScreen()
            }
            .show()
    }

    private fun showCoursewareManagementUploading(fileName: String) {
        currentScreen = Screen.CoursewareManagement
        val root = baseColumn().apply {
            setPadding(dp(28), dp(32), dp(28), dp(32))
        }
        root.addView(titleText("上传课件", 22f))
```

```
        val progressView = bodyText("正在上传：$fileName").apply {
            gravity = Gravity.CENTER
            layoutParams = LinearLayout.LayoutParams(
                ViewGroup.LayoutParams.MATCH_PARENT,
                ViewGroup.LayoutParams.WRAP_CONTENT
            ).apply { topMargin = dp(20) }
        }
        root.addView(progressView)
        showScreenView(root)
        statusText = progressView
    }

    private fun showCoursewareManagement() {
        currentScreen = Screen.CoursewareManagement
        val generation = authGeneration
        val root = baseColumn().apply {
            setPadding(dp(28), dp(32), dp(28), dp(32))
        }
        root.addView(titleText("课件管理", 22f))
        val statusView = bodyText("正在加载...").apply {
            gravity = Gravity.CENTER
            layoutParams = LinearLayout.LayoutParams(
                ViewGroup.LayoutParams.MATCH_PARENT,
                ViewGroup.LayoutParams.WRAP_CONTENT
            ).apply { topMargin = dp(20) }
        }
        root.addView(statusView)
        root.addView(secondaryButton("返回").apply {
            layoutParams = LinearLayout.LayoutParams(
                ViewGroup.LayoutParams.MATCH_PARENT,
                dp(50)
            ).apply { topMargin = dp(20) }
            setOnClickListener { showConnectScreen() }
        })
        showScreenView(root)

        Thread {
            runCatching {
                fetchServerCoursewareListBlocking()
            }.onSuccess { items ->
                runOnUiThread {
                    if (!isDestroyed && authGeneration == generation && currentScreen ==
                        Screen.CoursewareManagement) {
                        showCoursewareManagementList(items)
                    }
                }
            }.onFailure { error ->
                runOnUiThread {
                    if (!isDestroyed && authGeneration == generation) statusView.text =
                        "加载失败：${error.message}"
```

```
            }
        }
    }.start()
}

private fun showCoursewareManagementList(items: List<StoredCoursewareItem>) {
    showServerCoursewareList(items, management = true)
}

private val managementPickerLauncher = registerForActivityResult(
    ActivityResultContracts.GetContent()
) { uri: Uri? ->
    if (uri == null) return@registerForActivityResult
    managementUploadProgress(uri)
}

private fun previewManagementCourseware(item: StoredCoursewareItem) {
    if (item.linkUrl.isNotBlank()) {
        val uri = Uri.parse(item.linkUrl)
        if (uri.scheme !in listOf("http", "https")) { toast("无法预览该链接"); return }
        runCatching { startActivity(Intent(Intent.ACTION_VIEW, uri)) }
            .onFailure { toast("没有可打开此链接的浏览器") }
        return
    }
    val kind = mediaKindOf(item.url)
    if (kind == CastMediaKind.None) { toast("暂不支持预览此类课件"); return }
    val absolute = absoluteCoursewareUrl(item.url) ?: return
    managementPreview?.dismiss()
    managementPreview = CoursewarePreviewDialog(this, coursewareHttpClient, item.title,
        absolute,
        kind == CastMediaKind.Pdf,
        kind == CastMediaKind.Video, onDismiss = { managementPreview = null }).also { it.show()
            }
}

private fun uploadFromManagement() {
    managementPickerLauncher.launch("*/*")
}

private fun managementUploadProgress(uri: Uri) {
    val fileName = displayNameForUri(uri)
    showCoursewareManagementUploading(fileName)
    Thread {
        runCatching {
            uploadCoursewareBlocking(uri, fileName)
        }.onSuccess {
            runOnUiThread {
                toast("课件上传成功")
                showCoursewareManagement()
            }
```

```
            }.onFailure { error ->
                runOnUiThread {
                    toast(error.message ?: "课件上传失败")
                    showCoursewareManagement()
                }
            }
        }.start()
    }

    private fun confirmDeleteManagementCourseware(item: StoredCoursewareItem) {
        AlertDialog.Builder(this)
            .setTitle("删除课件")
            .setMessage("确定删除「${item.title}」吗？")
            .setNegativeButton("取消", null)
            .setPositiveButton("删除") { _, _ ->
                deleteManagementCourseware(item)
            }
            .show()
    }


    private fun deleteManagementCourseware(item: StoredCoursewareItem) {
        Thread {
            runCatching {
                deleteServerCoursewareBlocking(item)
            }.onSuccess {
                runOnUiThread {
                    toast("课件已删除")
                    showCoursewareManagement()
                }
            }.onFailure { error ->
                runOnUiThread {
                    toast(error.message ?: "删除失败")
                    showCoursewareManagement()
                }
            }
        }.start()
    }

    /** 从删除按钮向上找到所属 ScrollView，记录刷新前的滚动位置。 */
    private fun rememberCoursewareListScroll(view: View) {
        var parent = view.parent
        while (parent is View) {
            if (parent is ScrollView) {
                coursewareListScrollY = parent.scrollY
                return
            }
            parent = parent.parent
        }
    }
```

```
// 源文件：web/app.js（文件末尾节选）
  const width = annotationInkWidth(stroke, videoRect.width);
  context.save();
  if (stroke.isEraser) {
    // 标注画布透明（叠在视频上），板擦用 destination-out 真正擦除下层笔画
    context.globalCompositeOperation = 'destination-out';
    context.strokeStyle = '#000';
    context.fillStyle = '#000';
    context.lineWidth = width;
    context.lineCap = 'round';
    context.lineJoin = 'round';
  } else {
    context.strokeStyle = stroke.color;
    context.fillStyle = stroke.color;
    context.lineWidth = width;
    context.lineCap = 'round';
    context.lineJoin = 'round';
    if (stroke.mode === 'dashed') {
      context.setLineDash([width * 2.2, width * 1.6]);
    } else if (stroke.mode === 'dashdot') {
      context.setLineDash([width * 2.2, width * 1.2, 0.1, width * 1.2]);
    } else if (stroke.mode === 'highlighter') {
      context.lineWidth = width * 5;
      context.globalAlpha = 0.45;
    }
  }

  // 实线画笔走手写轮廓：线宽随压感 / 书写速度变化，起收笔收尖。
  // 板擦 / 荧光笔 / 虚线 / 直线工具保持等宽——它们不是"手写"，变宽反而怪。
  // 注意：这里不再区分有没有压感。红外屏 / 手指 / 电容笔的压力恒为 0.5，
  // 旧逻辑会把它降级成等宽二次曲线，教室里最常见的输入反而完全没有笔锋。
  const solidInk = !stroke.isEraser && (stroke.mode === 'solid' || !stroke.mode) &&
      !stroke.lineMode;
  if (solidInk && stroke.points.length >= 2) {
    drawFreehandStroke(context, stroke, canvasRect, videoRect, crop, width);
    context.restore();
    return;
  }

  let segmentStarted = false;
  let visiblePoints = 0;
  context.beginPath();

  for (const point of stroke.points) {
    const canvasPoint = sourcePointToCanvasPoint(point, canvasRect, videoRect, crop);
    if (!canvasPoint.visible) {
      if (segmentStarted) {
        context.stroke();
        context.beginPath();
        segmentStarted = false;
```

```
      }
      continue;
    }

    visiblePoints += 1;
    if (!segmentStarted) {
      context.moveTo(canvasPoint.x, canvasPoint.y);
      segmentStarted = true;
    } else {
      context.lineTo(canvasPoint.x, canvasPoint.y);
    }
  }

  if (segmentStarted) {
    context.stroke();
  }

  if (visiblePoints === 1) {
    const onlyPoint = stroke.points
      .map((point) => sourcePointToCanvasPoint(point, canvasRect, videoRect, crop))
      .find((point) => point.visible);
    if (onlyPoint) {
      context.beginPath();
      context.arc(onlyPoint.x, onlyPoint.y, (stroke.mode === 'highlighter' ? width * 5 : width) / 2,
          0, Math.PI * 2);
      context.fill();
    }
  }

  context.restore();
}

function sourcePointToCanvasPoint(point, canvasRect, videoRect, crop) {
  const videoX = (point.x - crop.x) / crop.width;
  const videoY = (point.y - crop.y) / crop.height;
  const visible = videoX >= 0 && videoX <= 1 && videoY >= 0 && videoY <= 1;
  return {
    x: videoRect.left - canvasRect.left + videoX * videoRect.width,
    y: videoRect.top - canvasRect.top + videoY * videoRect.height,
    visible
  };
}

function currentFrameCrop() {
  return {
    x: state.framePresentation.cropX,
    y: state.framePresentation.cropY,
    width: state.framePresentation.cropWidth,
    height: state.framePresentation.cropHeight
  };
```

```
}

function currentVideoContentRect() {
  if (isLocalVideoViewEnabled()) return lockedVideoContentRect();
  if (state.presentationMode === 'courseware') {
    // 图片课件：标注绑定到图片内容区域（其 transform 已随手机端视口放大/平移，
    // getBoundingClientRect 会返回变换后的实际区域，笔迹自动跟随图片移动）
    if (!elements.imagePlayerOverlay.hidden) {
      const imageRect = elements.coursewareImage.getBoundingClientRect();
      if (imageRect.width > 0 && imageRect.height > 0) {
        return imageRect;
      }
    }
    return elements.coursewareCanvas.getBoundingClientRect();
  }

  const elementRect = elements.remoteVideo.getBoundingClientRect();
  const videoWidth = elements.remoteVideo.videoWidth;
  const videoHeight = elements.remoteVideo.videoHeight;
  if (!videoWidth || !videoHeight || !elementRect.width || !elementRect.height) {
    return elementRect;
  }

  const fit = getComputedStyle(elements.remoteVideo).objectFit;
  const scale = fit === 'cover'
    ? Math.max(elementRect.width / videoWidth, elementRect.height / videoHeight)
    : Math.min(elementRect.width / videoWidth, elementRect.height / videoHeight);
  const width = videoWidth * scale;
  const height = videoHeight * scale;
  return {
    left: elementRect.left + (elementRect.width - width) / 2,
    top: elementRect.top + (elementRect.height - height) / 2,
    width,
    height
  };
}

function runCatching(callback) {
  try {
    callback();
  } catch (error) {
    return undefined;
  }
  return undefined;
}

function sendMessage(payload) {
  if (state.socket?.readyState === WebSocket.OPEN) {
    state.socket.send(JSON.stringify(payload));
  }
```

```
}

function showJoinView() {
  resetLockedVideoPan();
  hideDownloadButton();
  destroyCoursewareDocument(state.courseware);
  state.presentationMode = 'waiting';
  state.courseware = null;
  clearCoursewareCanvas();
  document.body.classList.remove('is-streaming');
  elements.imagePlayerOverlay.hidden = true;
  elements.joinView.hidden = false;
  elements.videoView.hidden = true;
  elements.remoteVideo.hidden = false;
  elements.coursewareCanvas.hidden = true;
  elements.panToolButton.hidden = true;
  elements.prevPageButton.hidden = true;
  elements.nextPageButton.hidden = true;
}

function showVideoView() {
  elements.videoStatus.classList.remove('courseware-error');
  coursewareLinkControls.then(links => links.clear());
  resetLockedVideoPan();
  state.presentationMode = 'video';
  setAnnotationTool('pen');
  elements.coursewareCanvas.hidden = true;
  elements.imagePlayerOverlay.hidden = true;
  elements.remoteVideo.hidden = false;
  document.body.classList.add('is-streaming');
  elements.joinView.hidden = true;
  elements.videoView.hidden = false;
  // Video/ZIP playback hides these; every annotatable view must restore both.
  elements.annotationCanvas.hidden = false;
  elements.annotationToolbar.hidden = false;
  elements.panToolButton.hidden = !isLocalVideoViewEnabled();
  elements.prevPageButton.hidden = true;
  elements.nextPageButton.hidden = true;
  elements.videoStatus.hidden = false;
  updateVideoPresentation();
}

function showCoursewareView() {
  state.presentationMode = 'courseware';
  showDownloadButtonIfAvailable();
  if (state.teacherToken && elements.coursewareDropdown) elements.coursewareDropdown.hidden = false;
  elements.remoteVideo.hidden = true;
  elements.imagePlayerOverlay.hidden = true;
  elements.coursewareCanvas.hidden = false;
  elements.videoView.dataset.orientation = 'landscape';
```

```
  elements.videoView.dataset.lockedZoomed = 'false';
  document.body.classList.add('is-streaming');
  elements.joinView.hidden = true;
  elements.videoView.hidden = false;
  elements.annotationCanvas.hidden = false;
  elements.annotationToolbar.hidden = false;
  elements.panToolButton.hidden = false;
  elements.prevPageButton.hidden = false;
  elements.nextPageButton.hidden = false;
  elements.videoStatus.hidden = true;
  updatePageNavButtons();
  resizeAnnotationCanvas();
  updateCoursewareConnectionIndicator();
}


function toggleFullscreen() {
  if (!document.fullscreenElement) {
    document.documentElement.requestFullscreen().catch(() => {});
  } else {
    document.exitFullscreen().catch(() => {});
  }
}

function updateFullscreenButton() {
  if (elements.fullscreenButton) {
    elements.fullscreenButton.textContent = document.fullscreenElement ? '退出全屏' : '全屏';
  }
}

// 首页「全屏」按钮文案：进入 / 退出都必须同步。
// 之前只在退出时重置，导致「首次点击自动全屏」后按钮仍写着"全屏"，
// 用户点它时被当成"退出全屏"，表现为"第一次点全屏反而退回未全屏"。
function updateHomeFullscreenButton() {
  if (elements.homeFullscreenButton) {
    elements.homeFullscreenButton.textContent = document.fullscreenElement ? '退出全屏' : '全屏';
  }
}

function handleCoursewareOriginal(message) {
  if (typeof message.originalUrl !== 'string' || !message.originalUrl) {
    return;
  }
  state.courseware = state.courseware || {};
  state.courseware.downloadOriginalUrl = message.originalUrl;
  state.downloadOriginalUrl = message.originalUrl;
  if (elements.coursewareDropdown) {
    elements.coursewareDropdown.hidden = false;
  }
}
```

```
function downloadOriginalFile() {
  const url = state.courseware?.downloadOriginalUrl || state.downloadOriginalUrl;
  if (!url) {
    return;
  }
  // 将相对路径转为完整 URL 并通过隐藏 a 标签触发下载
  const anchor = document.createElement('a');
  anchor.href = url;
  anchor.download = '';
  document.body.appendChild(anchor);
  anchor.click();
  document.body.removeChild(anchor);
}

function hideDownloadButton() {
  state.downloadOriginalUrl = null;
  if (elements.coursewareDropdown) {
    elements.coursewareDropdown.hidden = true;
    elements.coursewareDropdownMenu.classList.remove('is-open');
  }
}

function showDownloadButtonIfAvailable() {
  if (state.downloadOriginalUrl) {
    elements.coursewareDropdown.hidden = false;
  }
}

function setWaitingStatus(text) {
  elements.statusText.textContent = text;
  elements.videoStatus.textContent = text;
}

// -- 直接上课模式 --
async function loadTeacherCourseware() {
  if (!state.teacherToken) return;
  try {
    const res = await fetch('./api/courseware', {
      headers: { Authorization: `Bearer ${state.teacherToken}` }
    });
    if (!res.ok) throw new Error('获取课件失败');
    const data = await res.json();
    state.teacherCoursewareList = data.items || [];
    const items = state.teacherCoursewareList;
    if (!items.length) {
      elements.coursewareGrid.innerHTML = '<p style="color:var(--muted);
          text-align:center">暂无课件，请通过管理后台上传</p>';
    } else {
      elements.coursewareGrid.innerHTML = items.map((c, i) => {
```

```
        const fileName = c.fileName || c.url || '';
        const isVideo = /\.(mp4|mov|avi|webm|mkv|3gp)(\?|$)/i.test(fileName);
        const isLink = !!c.linkUrl;
        const isCourseware = /\.(ppt|pptx)(\?|$)/i.test(fileName);
        const isDocument = /\.(pdf|doc|docx|xls|xlsx)(\?|$)/i.test(fileName);
        let badge = '';
        if (isLink) badge = '<span class="courseware-link-badge">链接</span>';
        else if (isVideo) badge = '<span class="courseware-video-badge">视频</span>';
        else if (isCourseware) badge = '<span class="courseware-doc-badge">课件</span>';
        else if (isDocument) badge = '<span class="courseware-doc-badge">文档</span>';
        return `
        <div class="courseware-item" data-index="${i}">
          <div class="courseware-item-title">${badge}<span
              class="courseware-item-title-text">${escapeHtml(c.title)}</span></div>
          <div class="courseware-item-meta"><span class="courseware-item-meta-text">${escapeHtml(fil
              eName)} · ${isLink ? '外部链接' : formatSize(c.size)}</span></div>
        </div>
        `;
      }).join('');
      elements.coursewareGrid.querySelectorAll('.courseware-item').forEach((item) => {
        item.addEventListener('click', () => {
          const idx = parseInt(item.dataset.index, 10);
          const cw = state.teacherCoursewareList[idx];
          if (cw) openDirectCourseware(cw);
        });
      });
    }
    showPicker();
  } catch (err) {
    alert(err.message);
  }
}

function showPicker() {
  elements.coursewarePicker.hidden = false;
}

function showTeacherCoursewarePicker() {
  loadTeacherCourseware();
}

function openDirectCourseware(cw) {
  elements.coursewarePicker.hidden = true;
  elements.roomCode.textContent = '----';
  setWaitingStatus('');
  state.directTeach = true;
  state.coursewareFromViewer = true;

  // 链接类型课件：弹窗提示用户在大屏端打开
  if (cw.linkUrl) {
```

```
    showLinkPrompt(cw.title, cw.linkUrl);
    return;
  }

  // 如果有原文件下载地址，直接设置
  if (cw.originalUrl && cw.originalUrl !== cw.url) {
    state.downloadOriginalUrl = cw.originalUrl;
  } else {
    state.downloadOriginalUrl = null;
  }
  openCourseware({ url: cw.url, title: cw.title, page: 1, screen: 1 });

  // 通知手机端同步打开课件翻页页面
  sendMessage({
    type: 'viewer.courseware.open',
    token: state.teacherToken,
    url: cw.url,
    title: cw.title,
    page: 1,
    screen: 1
  });
}

function escapeHtml(str) {
  const div = document.createElement('div');
  div.textContent = str;
  return div.innerHTML;
}

function formatSize(b) {
  if (!b) return '0B';
  if (b < 1024 * 1024) return Math.round(b / 1024) + 'KB';
  return (b / 1024 / 1024).toFixed(1) + 'MB';
}

// ---- 链接课件提示弹窗 ----
function showLinkPrompt(title, linkUrl) {
  // 移除已有弹窗
  const existing = document.querySelector('.link-prompt-overlay');
  if (existing) existing.remove();

  const overlay = document.createElement('div');
  overlay.className = 'modal-overlay link-prompt-overlay';
  overlay.innerHTML = `
    <div class="modal-card" style="width:420px;text-align:center">
      <h3>🔗 ${escapeHtml(title)}</h3>
      <p style="color:var(--muted);margin:0.5rem 0 1.2rem;word-break:break-all;
          font-size:0.85rem">${escapeHtml(linkUrl)}</p>
      <p style="margin:0 0 1.2rem">教师已推送链接课件，是否在新窗口打开？</p>
      <div class="modal-actions" style="justify-content:center">
```

```
        <button class="btn-secondary" id="link-prompt-cancel">取消</button>
        <button class="btn-primary" id="link-prompt-open"
            style="margin-left:0.75rem">打开链接</button>
      </div>
    </div>
  `;
  document.body.appendChild(overlay);

  overlay.querySelector('#link-prompt-cancel').addEventListener('click', () => overlay.remove());
  overlay.querySelector('#link-prompt-open').addEventListener('click', () => {
    window.open(linkUrl, '_blank', 'noopener,noreferrer');
    overlay.remove();
  });
  // 点击遮罩关闭
  overlay.addEventListener('click', (e) => {
    if (e.target === overlay) overlay.remove();
  });
}

// ---- 黑板系统 ----
function toggleBlackboard(forceState) {
  const open = typeof forceState === 'boolean' ? forceState : !state.blackboard.active;
  if (open === state.blackboard.active) return;
  resetHandwritingTouches(elements.blackboardCanvas);

  state.blackboard.active = open;
  elements.blackboardOverlay.hidden = !open;
  elements.blackboardToggleButton.classList.toggle('is-active', open);

  if (open) {
    flushBlackboardActiveStrokes();
    state.blackboard.activeStrokes.clear();
    state.blackboard.panX = 0;
    state.blackboard.panY = 0;
    state.blackboard.scale = 1;
    state.blackboard._panActive = false;
    state.blackboard._pinch.active = false;
    state.blackboard._activePointers.clear();
    clearBlackboardSelection();
    setBlackboardTool('pen');
    updateBlackboardColorButtons();
    resizeBlackboardCanvas();
    renderBlackboard();
    updateBlackboardPageIndicator();
  }
}

function beginBlackboardStroke(event) {
  if (absorbPalmTouch(event)) return;
  if (!state.blackboard.active) return;
```

```
  if (handleHandwritingTouch(event, 'down', true)) return;

  if (state.blackboard.tool === 'hand' || state.blackboard.tool === 'select') {
    state.blackboard._activePointers.set(event.pointerId, { x: event.clientX, y: event.clientY });
  }

  // 手型工具：启动平移或双指缩放
  if (state.blackboard.tool === 'hand') {
    if (state.blackboard._activePointers.size >= 2) {
      abortBlackboardPan();
      startBlackboardPinch();
      return;
    }
    if (event.isPrimary) beginBlackboardPan(event);
    return;
  }

  // 圈选工具：启动套索绘制或拖拽
  if (state.blackboard.tool === 'select') {
    beginBlackboardSelect(event);
    return;
  }

  if (state.blackboard.activeStrokes.size) return;
  const point = blackboardPointerToPoint(event);
  if (!point) return;

  event.preventDefault();
  elements.blackboardCanvas.setPointerCapture(event.pointerId);

  const isEraser = state.blackboard.tool === 'eraser';
  const isLine = !isEraser && state.blackboard.lineMode;
  point.t = 0;
  point.p = pointerPressure(event);
  state.blackboard.activeStrokes.set(event.pointerId, {
    pointerId: event.pointerId,
    color: isEraser ? '#2c2f36' : state.blackboard.currentColor,
    width: isEraser ? state.blackboard.eraserWidth : state.annotations.penWidth + 2,
    isEraser: isEraser,
    mode: isEraser ? null : state.blackboard.mode,
    lineMode: isLine,
    points: [point],
    startedAt: event.timeStamp,
    startScreen: isLine ? { x: event.clientX, y: event.clientY } : null
  });
  scheduleBlackboardRedraw();
}

function continueBlackboardStroke(event) {
  if (absorbPalmTouch(event)) return;
```

```
if (!state.blackboard.active) return;
if (handleHandwritingTouch(event, 'move', true)) return;

if (state.blackboard._activePointers.has(event.pointerId)) {
  state.blackboard._activePointers.set(event.pointerId, { x: event.clientX, y: event.clientY });
}

// 圈选工具：套索绘制 或 拖拽移动
if (state.blackboard.tool === 'select') {
  continueBlackboardSelect(event);
  return;
}

// 手型捏合缩放中
if (state.blackboard._pinch.active) {
  continueBlackboardPinch();
  return;
}

// 手型平移中 → 检测是否出现第二指触发缩放
if (state.blackboard._panActive && state.blackboard._panPointerId === event.pointerId) {
  if (state.blackboard._activePointers.size >= 2) {
    abortBlackboardPan();
    startBlackboardPinch();
    return;
  }
  continueBlackboardPan(event);
  return;
}
const stroke = state.blackboard.activeStrokes.get(event.pointerId);
if (!stroke) return;
event.preventDefault();
const point = blackboardPointerToPoint(event);
if (!point) return;
const lastPoint = stroke.points.at(-1);
if (stroke.lineMode && lastPoint && Math.hypot(point.x - lastPoint.x, point.y - lastPoint.y) <
    0.001) return;
if (stroke.lineMode) {
  // 直线模式：只保留起点与当前点，水平/竖直自动吸附，实时显示长度与夹角
  const { storagePoint, screenPoint } = computeLineSnap(stroke.points[0], point,
      stroke.startScreen, event);
  stroke.points = [stroke.points[0], storagePoint];
  updateAngleIndicator(stroke.startScreen, screenPoint);
} else {
  for (const sample of handwritingSamples(event)) {
    const next = blackboardPointerToPoint(sample);
    if (!next) continue;
    const last = stroke.points.at(-1);
    if (last && Math.hypot(next.x - last.x, next.y - last.y) * state.blackboard.scale < 0.5)
        continue;
```

```
      next.t = Math.max(0, Math.round(sample.timeStamp - (stroke.startedAt ?? sample.timeStamp)));
      next.p = pointerPressure(sample);
      stroke.points.push(next);
    }
  }
  scheduleBlackboardRedraw();
}

function finishBlackboardStroke(event) {
  noteStylus(event);
  if (!state.blackboard.active) return;
  if (event.type === 'pointerup' && state.blackboard.activeStrokes.has(event.pointerId)) {
    continueBlackboardStroke(event);
  }
  if (handleHandwritingTouch(event, 'up', true)) return;

  // 清理指针缓存
  state.blackboard._activePointers.delete(event.pointerId);

  // 圈选工具：套索完成 或 拖拽结束
  if (state.blackboard.tool === 'select') {
    finishBlackboardSelect(event);
    return;
  }

  // 捏合缩放结束
  if (state.blackboard._pinch.active) {
    endBlackboardPinch();
    return;
  }

  // 手型平移结束
  if (state.blackboard._panActive && state.blackboard._panPointerId === event.pointerId) {
    finishBlackboardPan(event);
    return;
  }
  completeBlackboardStroke(event);
}

function completeBlackboardStroke(event, releaseCapture = true) {
  const stroke = state.blackboard.activeStrokes.get(event.pointerId);
  if (!stroke) return;
  event.preventDefault();
  if (stroke.points.length > 0) {
    const pages = state.blackboard.pages;
    pages[state.blackboard.currentPage].strokes.push({
      color: stroke.color,
      width: stroke.width,
      isEraser: !!stroke.isEraser,
      mode: stroke.mode,
```

```
      lineMode: !!stroke.lineMode,
      points: stroke.points
    });
  }
  state.blackboard.activeStrokes.delete(event.pointerId);
  if (releaseCapture) runCatching(() => elements.blackboardCanvas.releasePointerCapture(event.pointe
      rId));
  if (stroke.lineMode) hideAngleIndicator();

  renderBlackboard();
  updateBlackboardPageIndicator();
}

const LINE_SNAP_THRESHOLD = 5; // 水平/竖直吸附阈值（度）

// 直线吸附：根据屏幕角度判断是否吸附到水平/竖直，返回存储坐标点和屏幕坐标点
function computeLineSnap(startPoint, point, startScreen, event) {
  const dx = event.clientX - startScreen.x;
  const dy = event.clientY - startScreen.y;
  const angle = Math.atan2(-dy, dx) * 180 / Math.PI;
  let storagePoint = point;
  let screenPoint = { x: event.clientX, y: event.clientY };
  // 水平吸附：角度接近 0° 或 ±180°
  if (Math.abs(angle) < LINE_SNAP_THRESHOLD || Math.abs(angle) > 180 - LINE_SNAP_THRESHOLD) {
    storagePoint = { x: point.x, y: startPoint.y };
    screenPoint = { x: event.clientX, y: startScreen.y };
  } else if (Math.abs(Math.abs(angle) - 90) < LINE_SNAP_THRESHOLD) {
    // 竖直吸附：角度接近 ±90°
    storagePoint = { x: startPoint.x, y: point.y };
    screenPoint = { x: startScreen.x, y: event.clientY };
  }
  return { storagePoint, screenPoint };
}

function updateAngleIndicator(startScreen, screenPoint) {
  if (!startScreen || !elements.angleIndicator) return;
  const dx = screenPoint.x - startScreen.x;
  const dy = screenPoint.y - startScreen.y;
  const length = Math.hypot(dx, dy);
  if (length < 2) return;
  // 与水平方向的夹角（屏幕 y 轴向下，取负使向上为正），范围 -180°~180°
  const angle = Math.atan2(-dy, dx) * 180 / Math.PI;
  elements.angleValue.textContent = `${angle.toFixed(1)}°`;
  elements.lengthValue.textContent = `${Math.round(length)}px`;
  elements.angleIndicator.style.left = `${screenPoint.x}px`;
  elements.angleIndicator.style.top = `${screenPoint.y}px`;
  elements.angleIndicator.classList.add('is-visible');
}

function hideAngleIndicator() {
```

```
  if (elements.angleIndicator) {
    elements.angleIndicator.classList.remove('is-visible');
  }
}

function blackboardPointerToPoint(event) {
  const rect = elements.blackboardCanvas.getBoundingClientRect();
  if (rect.width <= 0 || rect.height <= 0) return null;
  const scale = state.blackboard.scale;
  // 屏幕坐标 → 世界坐标（除以缩放 + 视口偏移）
  const x = (event.clientX - rect.left) / scale + state.blackboard.panX;
  const y = (event.clientY - rect.top) / scale + state.blackboard.panY;
  return { x, y };
}

function resizeBlackboardCanvas() {
  const canvas = elements.blackboardCanvas;
  const rect = canvas.getBoundingClientRect();
  const ratio = Math.min(window.devicePixelRatio || 1, 2);
  const width = Math.max(1, Math.round(rect.width * ratio));
  const height = Math.max(1, Math.round(rect.height * ratio));
  if (canvas.width !== width || canvas.height !== height) {
    canvas.width = width;
    canvas.height = height;
  }
}

function renderBlackboard(rebuild = true) {
  resizeBlackboardCanvas();
  const canvas = elements.blackboardCanvas;
  const ctx = canvas.getContext('2d');
  const rect = canvas.getBoundingClientRect();
  const ratio = Math.min(window.devicePixelRatio || 1, 2);
  const panX = state.blackboard.panX;
  const panY = state.blackboard.panY;
  const scale = state.blackboard.scale;
  const s = scale * ratio;

  const page = state.blackboard.pages[state.blackboard.currentPage];
  const key = JSON.stringify([canvas.width, canvas.height, ratio, panX, panY, scale,
      state.blackboard.currentPage]);
  prepareInkCache(blackboardInkCache, canvas, key, rebuild, base => {
    base.setTransform(s, 0, 0, s, -panX * s, -panY * s);
    base.fillStyle = '#2c2f36';
    base.fillRect(panX, panY, rect.width / scale, rect.height / scale);
    drawBlackboardGrid(base, rect);
    for (const stroke of page?.strokes || []) drawBlackboardStroke(base, stroke);
  });
  ctx.setTransform(1, 0, 0, 1, 0, 0);
  ctx.clearRect(0, 0, canvas.width, canvas.height);
```

```
  ctx.drawImage(blackboardInkCache.canvas, 0, 0);
  ctx.setTransform(s, 0, 0, s, -panX * s, -panY * s);
  for (const stroke of state.blackboard.activeStrokes.values()) {
    drawBlackboardStroke(ctx, stroke);
  }

  // 渲染圈选
  renderBlackboardSelection(ctx);
}

function drawBlackboardStroke(ctx, stroke) {
  if (stroke.points.length === 0) return;
  ctx.save();
  ctx.strokeStyle = stroke.color;
  ctx.fillStyle = stroke.color;
  ctx.lineWidth = stroke.width;
  ctx.lineCap = 'round';
  ctx.lineJoin = 'round';
  if (!stroke.isEraser) {
    if (stroke.mode === 'dashed') {
      ctx.setLineDash([stroke.width * 2.2, stroke.width * 1.6]);
    } else if (stroke.mode === 'dashdot') {
      ctx.setLineDash([stroke.width * 2.2, stroke.width * 1.2, 0.1, stroke.width * 1.2]);
    } else if (stroke.mode === 'highlighter') {
      ctx.lineWidth = stroke.width * 5;
      ctx.globalAlpha = 0.45;
    }
  }

  // 黑板坐标是世界像素，和课件标注走同一套手写管线（去重 / 平滑 / 速度笔锋 / 起收笔收尖）。
  // 以前这里用 inkWidths + traceInkOutline：忽略压力、斜接多边形又不做抽稀，
  // 采样一密就是锯齿和毛刺——这是《红外触控书写优化》里最容易看出差距的一条路径。
  const solidInk = !stroke.isEraser && (stroke.mode === 'solid' || !stroke.mode) &&
      !stroke.lineMode;
  if (solidInk && stroke.points.length >= 2) {
    drawBlackboardFreehand(ctx, stroke);
    ctx.restore();
    return;
  }

  ctx.beginPath();
  ctx.moveTo(stroke.points[0].x, stroke.points[0].y);
  for (let i = 1; i < stroke.points.length; i++) {
    ctx.lineTo(stroke.points[i].x, stroke.points[i].y);
  }
  ctx.stroke();

  if (stroke.points.length === 1) {
    ctx.beginPath();
    const r = (!stroke.isEraser && stroke.mode === 'highlighter') ? stroke.width * 5 : stroke.width;
```

```
    ctx.arc(stroke.points[0].x, stroke.points[0].y, r / 2, 0, Math.PI * 2);
    ctx.fill();
  }
  ctx.restore();
}

function drawBlackboardGrid(ctx, rect) {
  const panX = state.blackboard.panX;
  const panY = state.blackboard.panY;
  const scale = state.blackboard.scale;
  const gridSize = 48;
  const dotRadius = 1.2;
  const startX = Math.floor(panX / gridSize) * gridSize;
  const startY = Math.floor(panY / gridSize) * gridSize;
  const endX = panX + rect.width / scale;
  const endY = panY + rect.height / scale;

  ctx.fillStyle = 'rgba(255,255,255,0.055)';
  for (let x = startX; x <= endX; x += gridSize) {
    for (let y = startY; y <= endY; y += gridSize) {
      ctx.beginPath();
      ctx.arc(x, y, dotRadius, 0, Math.PI * 2);
      ctx.fill();
    }
  }
}

function flushBlackboardActiveStrokes() {
  const pages = state.blackboard.pages;
  const page = pages[state.blackboard.currentPage];
  if (!page) return;
  for (const stroke of state.blackboard.activeStrokes.values()) {
    if (stroke.points.length > 0) {
      page.strokes.push({ color: stroke.color, width: stroke.width, isEraser: !!stroke.isEraser,
          mode: stroke.mode, points: stroke.points });
    }
  }
  state.blackboard.activeStrokes.clear();
}

function navigateBlackboardPage(delta) {
  flushBlackboardActiveStrokes();
  clearBlackboardSelection();
  const pages = state.blackboard.pages;
  const newPage = state.blackboard.currentPage + (delta < 0 ? -1 : 1);
  if (newPage < 0 || newPage >= pages.length) return;
  state.blackboard.currentPage = newPage;
  state.blackboard.activeStrokes.clear();
  state.blackboard.panX = 0;
  state.blackboard.panY = 0;
```

```
  renderBlackboard();
  updateBlackboardPageIndicator();
}

function addBlackboardPage() {
  flushBlackboardActiveStrokes();
  clearBlackboardSelection();
  const pages = state.blackboard.pages;
  // 在当前页之后插入新页
  const insertAt = state.blackboard.currentPage + 1;
  pages.splice(insertAt, 0, { strokes: [] });
  state.blackboard.currentPage = insertAt;
  state.blackboard.activeStrokes.clear();
  state.blackboard.panX = 0;
  state.blackboard.panY = 0;
  renderBlackboard();
  updateBlackboardPageIndicator();
}

function deleteBlackboardPage() {
  const pages = state.blackboard.pages;
  if (pages.length <= 1) return; // 至少保留一页
  flushBlackboardActiveStrokes();
  clearBlackboardSelection();
  const idx = state.blackboard.currentPage;
  pages.splice(idx, 1);
  if (state.blackboard.currentPage >= pages.length) {
    state.blackboard.currentPage = pages.length - 1;
  }
  state.blackboard.activeStrokes.clear();
  state.blackboard.panX = 0;
  state.blackboard.panY = 0;
  renderBlackboard();
  updateBlackboardPageIndicator();
}

function updateBlackboardPageIndicator() {
  const pages = state.blackboard.pages;
  const cur = state.blackboard.currentPage + 1;
  const page = pages[state.blackboard.currentPage];
  const hasStrokes = page && page.strokes.length > 0;
  elements.blackboardPageIndicator.textContent = `第 ${cur} / ${pages.length} 页`;
  elements.blackboardPrevPageButton.disabled = state.blackboard.currentPage <= 0;
  elements.blackboardNextPageButton.disabled = state.blackboard.currentPage >= pages.length - 1;
  elements.blackboardDelPageButton.disabled = pages.length <= 1;
  elements.blackboardUndoButton.disabled = !hasStrokes && !page?.cleared?.length;
  elements.blackboardClearButton.disabled = !hasStrokes;
}

function setBlackboardTool(tool) {
```

```
  // 切换工具时清除旧圈选状态
  if (tool !== 'select') clearBlackboardSelection();

  if (tool === 'eraser') {
    state.blackboard.tool = 'eraser';
  } else if (tool === 'hand') {
    state.blackboard.tool = 'hand';
  } else if (tool === 'select') {
    state.blackboard.tool = 'select';
  } else {
    state.blackboard.tool = 'pen';
  }

  const isEraser = state.blackboard.tool === 'eraser';
  const isHand = state.blackboard.tool === 'hand';
  const isPen = state.blackboard.tool === 'pen';
  const isSelect = state.blackboard.tool === 'select';

  elements.blackboardCanvas.classList.toggle('is-eraser', isEraser);
  elements.blackboardCanvas.classList.toggle('is-hand-tool', isHand);
  elements.blackboardCanvas.classList.toggle('is-select-tool', isSelect);
  elements.blackboardEraserButton.classList.toggle('is-active', isEraser);
  elements.blackboardHandButton.classList.toggle('is-active', isHand);
  elements.blackboardSelectButton.classList.toggle('is-active', isSelect);

  // 删除按钮仅在圈选工具有选中内容时显示
  elements.blackboardDeleteSelButton.style.display = (isSelect &&
      state.blackboard.selection.confirmedIndices.length > 0) ? '' : 'none';

  // 非画笔模式只移除调色盘选中圈，调色盘保持可见
  if (isPen) {
    updateBlackboardColorButtons();
  } else {
    elements.blackboardColorButtons.forEach((btn) => btn.classList.remove('is-active'));
  }

  if (!isEraser) {
    elements.blackboardEraserCursor.style.display = 'none';
  } else {
    elements.blackboardEraserCursor.style.display = '';
  }
}

function toggleBlackboardHand() {
  const newTool = state.blackboard.tool === 'hand' ? 'pen' : 'hand';
  setBlackboardTool(newTool);
}

function undoBlackboardStroke() {
  flushBlackboardActiveStrokes();
```

```
  clearBlackboardSelection();
  const page = state.blackboard.pages[state.blackboard.currentPage];
  if (page && page.strokes.length > 0) {
    page.strokes.pop();
  } else if (page?.cleared?.length) {
    page.strokes = page.cleared.pop();
  }
  renderBlackboard();
  updateBlackboardPageIndicator();
}

function clearBlackboard() {
  flushBlackboardActiveStrokes();
  clearBlackboardSelection();
  const page = state.blackboard.pages[state.blackboard.currentPage];
  if (page) {
    if (page.strokes.length) (page.cleared ||= []).push(page.strokes);
    page.strokes = [];
  }
  state.blackboard.activeStrokes.clear();
  renderBlackboard();
  updateBlackboardPageIndicator();
}

function beginBlackboardPan(event) {
  if (!state.blackboard.active) return;
  event.preventDefault();
  elements.blackboardCanvas.setPointerCapture(event.pointerId);
  state.blackboard._panActive = true;
  state.blackboard._panPointerId = event.pointerId;
  state.blackboard._panStartX = event.clientX;
  state.blackboard._panStartY = event.clientY;
  state.blackboard._panStartPanX = state.blackboard.panX;
  state.blackboard._panStartPanY = state.blackboard.panY;
  elements.blackboardCanvas.classList.add('is-panning');
}

function continueBlackboardPan(event) {
  if (!state.blackboard._panActive) return;
  event.preventDefault();
  const scale = state.blackboard.scale;
  const dx = (event.clientX - state.blackboard._panStartX) / scale;
  const dy = (event.clientY - state.blackboard._panStartY) / scale;
  state.blackboard.panX = state.blackboard._panStartPanX - dx;
  state.blackboard.panY = state.blackboard._panStartPanY - dy;
  renderBlackboard();
}

function finishBlackboardPan(event) {
  if (!state.blackboard._panActive) return;
```

```
  event.preventDefault();
  state.blackboard._panActive = false;
  state.blackboard._panPointerId = null;
  runCatching(() => elements.blackboardCanvas.releasePointerCapture(event.pointerId));
  elements.blackboardCanvas.classList.remove('is-panning');
}

// 取消当前平移（不清除指针捕捉，由后续操作接管）
function abortBlackboardPan() {
  if (!state.blackboard._panActive) return;
  state.blackboard._panActive = false;
  state.blackboard._panPointerId = null;
  elements.blackboardCanvas.classList.remove('is-panning');
}

function pointerDist(a, b) {
  return Math.hypot(b.x - a.x, b.y - a.y);
}

function pinchCenter(a, b) {
  return { x: (a.x + b.x) / 2, y: (a.y + b.y) / 2 };
}

// 启动双指缩放
function startBlackboardPinch() {
  const pointers = [...state.blackboard._activePointers.values()];
  if (pointers.length < 2) return;
  const r = elements.blackboardCanvas.getBoundingClientRect();
  const dist = pointerDist(pointers[0], pointers[1]);
  const center = pinchCenter(pointers[0], pointers[1]);

  state.blackboard._pinch.active = true;
  state.blackboard._pinch.startDist = dist > 0 ? dist : 1;
  state.blackboard._pinch.startScale = state.blackboard.scale;
  state.blackboard._pinch.centerX = center.x - r.left;
  state.blackboard._pinch.centerY = center.y - r.top;
}

// 持续双指缩放（以捏合中心为焦点）
function continueBlackboardPinch() {
  const pointers = [...state.blackboard._activePointers.values()];
  if (pointers.length < 2) {
    endBlackboardPinch();
    return;
  }
  const r = elements.blackboardCanvas.getBoundingClientRect();
  const dist = pointerDist(pointers[0], pointers[1]);
  const center = pinchCenter(pointers[0], pointers[1]);
  if (dist < 0.5) return;
```

```
  const pinch = state.blackboard._pinch;
  const ratio = dist / pinch.startDist;
  const newScale = clamp(pinch.startScale * ratio, state.blackboard.MIN_SCALE,
      state.blackboard.MAX_SCALE);

  // 以捏合中心为焦点缩放：世界坐标中心点保持不变
  const screenCx = center.x - r.left;
  const screenCy = center.y - r.top;
  const oldScale = state.blackboard.scale;
  if (Math.abs(newScale - oldScale) > 0.001) {
    state.blackboard.panX += screenCx / oldScale - screenCx / newScale;
    state.blackboard.panY += screenCy / oldScale - screenCy / newScale;
    state.blackboard.scale = newScale;
    renderBlackboard();
  }
}

// 结束双指缩放
function endBlackboardPinch() {
  const wasActive = state.blackboard._pinch.active;
  state.blackboard._pinch.active = false;
  state.blackboard._pinch.startDist = 0;
  if (!wasActive) return;
  // 若仍有剩余单指，启动平移接续
  if (state.blackboard._activePointers.size === 1 && state.blackboard.tool === 'hand') {
    const [pointerId, pt] = [...state.blackboard._activePointers][0];
    elements.blackboardCanvas.setPointerCapture(pointerId);
    state.blackboard._panActive = true;
    state.blackboard._panPointerId = pointerId;
    state.blackboard._panStartX = pt.x;
    state.blackboard._panStartY = pt.y;
    state.blackboard._panStartPanX = state.blackboard.panX;
    state.blackboard._panStartPanY = state.blackboard.panY;
    elements.blackboardCanvas.classList.add('is-panning');
  }
}

function setBlackboardMode(mode) {
  state.blackboard.mode = mode;
  updateBlackboardModeButtons();
}

function updateBlackboardModeButtons() {
  if (!elements.blackboardModeMenu) return;
  elements.blackboardModeMenu.querySelectorAll('.toolbar-dropdown-item[data-mode]').forEach((button)
      => {
    const active = button.dataset.mode === state.blackboard.mode;
    button.classList.toggle('is-active', active);
    button.setAttribute('aria-pressed', String(active));
  });
```

```
}

function setBlackboardColor(color) {
  if (!color) return;
  state.blackboard.currentColor = color;
  // 选颜色自动切回画笔
  setBlackboardTool('pen');
  updateBlackboardColorButtons();
  updateModeMenuIconColor(elements.blackboardModeMenu, color);
}

function updateBlackboardColorButtons() {
  elements.blackboardColorButtons.forEach((btn) => {
    btn.classList.toggle('is-active', btn.dataset.color === state.blackboard.currentColor);
  });
  const extraColor = ['#E87500', '#7B2CBF', '#222222', '#FFFFFF']
      .includes(state.blackboard.currentColor);
  elements.blackboardMoreColorsButton.classList.toggle(
    'is-active',
    state.blackboard.currentColor === '#00843D' || extraColor
  );
  elements.blackboardMoreColorsButton.style.setProperty(
    '--annotation-selection-ring',
    extraColor ? state.blackboard.currentColor : '#FFFFFF'
  );
}

function toggleBlackboardEraser() {
  const newTool = state.blackboard.tool === 'eraser' ? 'pen' : 'eraser';
  setBlackboardTool(newTool);
}

// ========== 圈选系统 ==========

function clearBlackboardSelection() {
  const sel = state.blackboard.selection;
  sel.lassoPoints = [];
  sel.drawing = false;
  sel.confirmedIndices = [];
  sel.bbox = null;
  sel.dragging = false;
  sel.dragStartWX = 0;
  sel.dragStartWY = 0;
  sel.dragSnapshot = null;
  elements.blackboardDeleteSelButton.style.display = 'none';
}

function toggleBlackboardSelect() {
  flushBlackboardActiveStrokes();
  if (state.blackboard.tool === 'select') {
```

```
    setBlackboardTool('pen');
  } else {
    setBlackboardTool('select');
  }
}

function beginBlackboardSelect(event) {
  const sel = state.blackboard.selection;

  // 如果已有确认的选中内容，尝试拖拽
  if (sel.confirmedIndices.length > 0) {
    const point = blackboardPointerToPoint(event);
    if (!point) return;
    // 判断点击是否落在已选区域的包围盒内
    if (sel.bbox && point.x >= sel.bbox.minX - 20 && point.x <= sel.bbox.maxX + 20
        && point.y >= sel.bbox.minY - 20 && point.y <= sel.bbox.maxY + 20) {
      // 启动拖拽
      event.preventDefault();
      elements.blackboardCanvas.setPointerCapture(event.pointerId);
      sel.dragging = true;
      sel.dragStartWX = point.x;
      sel.dragStartWY = point.y;
      sel.dragSnapshot = deepCopyStrokesForSelection();
      return;
    }
    // 点击在选区外 → 取消选中
    if (sel.confirmedIndices.length > 0) {
      clearBlackboardSelection();
      renderBlackboard();
      return;
    }
  }

  // 否则开始绘制套索
  const point = blackboardPointerToPoint(event);
  if (!point) return;

  // 清空旧结果
  sel.confirmedIndices = [];
  sel.bbox = null;
  sel.dragSnapshot = null;
  elements.blackboardDeleteSelButton.style.display = 'none';

  event.preventDefault();
  elements.blackboardCanvas.setPointerCapture(event.pointerId);
  sel.drawing = true;
  sel.lassoPoints = [point];
  renderBlackboard();
}
```

```
function continueBlackboardSelect(event) {
  const sel = state.blackboard.selection;

  // 拖拽移动中
  if (sel.dragging && sel.dragSnapshot) {
    event.preventDefault();
    const point = blackboardPointerToPoint(event);
    if (!point) return;
    const dx = point.x - sel.dragStartWX;
    const dy = point.y - sel.dragStartWY;
    applyDragOffsetToSnapshot(sel.dragSnapshot, dx, dy);
    sel.dragStartWX = point.x;
    sel.dragStartWY = point.y;
    renderBlackboard();
    return;
  }

  // 套索绘制中
  if (!sel.drawing) return;
  event.preventDefault();
  const point = blackboardPointerToPoint(event);
  if (!point) return;

  // 距离阈值，避免点过密
  const last = sel.lassoPoints[sel.lassoPoints.length - 1];
  if (last && Math.hypot(point.x - last.x, point.y - last.y) < 5) return;

  sel.lassoPoints.push(point);
  renderBlackboard();
}

function finishBlackboardSelect(event) {
  const sel = state.blackboard.selection;

  // 拖拽结束：提交
  if (sel.dragging) {
    sel.dragging = false;
    runCatching(() => elements.blackboardCanvas.releasePointerCapture(event.pointerId));
    commitDragSelection(sel.dragSnapshot);
    sel.dragSnapshot = null;
    renderBlackboard();
    return;
  }

  // 套索完成
  if (!sel.drawing) return;
  sel.drawing = false;
  runCatching(() => elements.blackboardCanvas.releasePointerCapture(event.pointerId));

  // 需要至少 3 个点才能形成闭合区域
```

```
  if (sel.lassoPoints.length < 3) {
    sel.lassoPoints = [];
    clearBlackboardSelection();
    renderBlackboard();
    return;
  }

  // 闭合套索（首尾相连）
  const poly = [...sel.lassoPoints];
  if (poly.length > 2) {
    poly.push({ x: poly[0].x, y: poly[0].y });
  }

  // 检测哪些笔画在套索内
  flushBlackboardActiveStrokes();
  const page = state.blackboard.pages[state.blackboard.currentPage];
  if (!page) {
    sel.lassoPoints = [];
    renderBlackboard();
    return;
  }

  const indices = [];
  for (let i = 0; i < page.strokes.length; i++) {
    // 跳过板擦笔迹本身
    if (page.strokes[i].isEraser) continue;
    // 跳过已被后续板擦笔迹完全覆盖的笔迹
    if (isStrokeFullyErased(i, page)) continue;
    if (isStrokeInPolygon(page.strokes[i], poly)) {
      indices.push(i);
    }
  }

  if (indices.length > 0) {
    sel.confirmedIndices = indices;
    sel.bbox = computeBBox(indices.map(i => page.strokes[i]));
    elements.blackboardDeleteSelButton.style.display = '';
  } else {
    clearBlackboardSelection();
  }

  sel.lassoPoints = [];
  renderBlackboard();
}

// 判断笔迹是否被后续的板擦笔迹完全覆盖（视觉上已被擦除）
function isStrokeFullyErased(strokeIndex, page) {
  const stroke = page.strokes[strokeIndex];
  if (!stroke || !stroke.points || stroke.points.length === 0) return true;
```

```
  // 遍历当前笔迹之后的所有板擦笔迹
  for (let j = strokeIndex + 1; j < page.strokes.length; j++) {
    const eraser = page.strokes[j];
    if (!eraser.isEraser) continue;
    if (!eraser.points || eraser.points.length < 2) continue;

    const eraserRadius = (eraser.width || 40) / 2;

    // 检查当前笔迹的所有点是否都在板擦覆盖范围内
    let allCovered = true;
    for (const pt of stroke.points) {
      if (!isPointWithinEraserPath(pt, eraser.points, eraserRadius)) {
        allCovered = false;
        break;
      }
    }
    if (allCovered) return true;
  }
  return false;
}

// 判断一个点是否在板擦路径覆盖范围内（点到任意线段的距离 <= 板擦半径）
function isPointWithinEraserPath(pt, eraserPoints, radius) {
  for (let k = 0; k < eraserPoints.length - 1; k++) {
    const seg = { x1: eraserPoints[k].x, y1: eraserPoints[k].y, x2: eraserPoints[k + 1].x, y2:
        eraserPoints[k + 1].y };
    if (pointToSegmentDistSq(pt.x, pt.y, seg.x1, seg.y1, seg.x2, seg.y2) <= radius * radius) {
      return true;
    }
  }
  // 单独检查最后一个孤立点（如果板擦只有一个点）
  if (eraserPoints.length === 1) {
    const dx = pt.x - eraserPoints[0].x;
    const dy = pt.y - eraserPoints[0].y;
    return dx * dx + dy * dy <= radius * radius;
  }
  return false;
}

// 点 (px,py) 到线段 (x1,y1)-(x2,y2) 的平方距离
function pointToSegmentDistSq(px, py, x1, y1, x2, y2) {
  const dx = x2 - x1;
  const dy = y2 - y1;
  const lenSq = dx * dx + dy * dy;
  if (lenSq === 0) {
    const ex = px - x1;
    const ey = py - y1;
    return ex * ex + ey * ey;
  }
  let t = ((px - x1) * dx + (py - y1) * dy) / lenSq;
```

```
  t = Math.max(0, Math.min(1, t));
  const projX = x1 + t * dx;
  const projY = y1 + t * dy;
  const ex = px - projX;
  const ey = py - projY;
  return ex * ex + ey * ey;
}

// 射线法判断点是否在多边形内
function isPointInPolygon(px, py, polygon) {
  let inside = false;
  const n = polygon.length;
  for (let i = 0, j = n - 1; i < n; j = i++) {
    const xi = polygon[i].x, yi = polygon[i].y;
    const xj = polygon[j].x, yj = polygon[j].y;
    if ((yi > py) !== (yj > py) && px < (xj - xi) * (py - yi) / (yj - yi) + xi) {
      inside = !inside;
    }
  }
  return inside;
}

// 判断笔画是否与套索有交集（至少一个点落在多边形内）
function isStrokeInPolygon(stroke, polygon) {
  if (!stroke.points || stroke.points.length === 0) return false;
  for (const p of stroke.points) {
    if (isPointInPolygon(p.x, p.y, polygon)) return true;
  }
  return false;
}

// 计算笔画集合的包围盒
function computeBBox(strokes) {
  if (!strokes || strokes.length === 0) return null;
  let minX = Infinity, minY = Infinity, maxX = -Infinity, maxY = -Infinity;
  for (const s of strokes) {
    if (!s.points) continue;
    for (const p of s.points) {
      if (p.x < minX) minX = p.x;
      if (p.y < minY) minY = p.y;
      if (p.x > maxX) maxX = p.x;
      if (p.y > maxY) maxY = p.y;
    }
  }
  return { minX, minY, maxX, maxY };
}

// 深拷贝选中的笔画（用于拖拽预览）
function deepCopyStrokesForSelection() {
  const page = state.blackboard.pages[state.blackboard.currentPage];
```

```
  if (!page) return [];
  return state.blackboard.selection.confirmedIndices.map(i => ({
    color: page.strokes[i].color,
    width: page.strokes[i].width,
    isEraser: !!page.strokes[i].isEraser,
    mode: page.strokes[i].mode,
    points: page.strokes[i].points.map(p => ({ x: p.x, y: p.y })),
  }));
}

// 对拖拽快照中的所有笔画点位施加偏移
function applyDragOffsetToSnapshot(snapshot, dx, dy) {
  for (const s of snapshot) {
    for (const p of s.points) {
      p.x += dx;
      p.y += dy;
    }
  }
}

// 提交拖拽：从原始 page 中删除旧笔画，插入偏移后的新笔画
function commitDragSelection(snapshot) {
  if (!snapshot || snapshot.length === 0) return;
  const page = state.blackboard.pages[state.blackboard.currentPage];
  if (!page) return;
  const indices = [...state.blackboard.selection.confirmedIndices].sort((a, b) => b - a);
  for (const i of indices) {
    page.strokes.splice(i, 1);
  }
  // 追加新笔画
  for (const s of snapshot) {
    page.strokes.push({
      color: s.color,
      width: s.width,
      isEraser: !!s.isEraser,
      mode: s.mode,
      points: s.points.map(p => ({ x: p.x, y: p.y })),
    });
  }
  // 新确认的索引是刚追加的这些
  state.blackboard.selection.confirmedIndices = [];
  for (let i = page.strokes.length - snapshot.length; i < page.strokes.length; i++) {
    state.blackboard.selection.confirmedIndices.push(i);
  }
  state.blackboard.selection.bbox = computeBBox(snapshot);
  if (state.blackboard.selection.confirmedIndices.length > 0) {
    elements.blackboardDeleteSelButton.style.display = '';
  }
}
```

```
// 删除圈选的笔画
function deleteBlackboardSelection() {
  flushBlackboardActiveStrokes();
  const page = state.blackboard.pages[state.blackboard.currentPage];
  if (!page) return;
  const indices = [...state.blackboard.selection.confirmedIndices].sort((a, b) => b - a);
  for (const i of indices) {
    page.strokes.splice(i, 1);
  }
  clearBlackboardSelection();
  renderBlackboard();
  updateBlackboardPageIndicator();
}

// 渲染圈选视觉
function renderBlackboardSelection(ctx) {
  const sel = state.blackboard.selection;
  const page = state.blackboard.pages[state.blackboard.currentPage];

  // 1. 渲染正在绘制的套索
  if (sel.drawing && sel.lassoPoints.length >= 2) {
    ctx.save();
    ctx.strokeStyle = 'rgba(56, 189, 248, 0.9)';
    ctx.lineWidth = 2;
    ctx.lineCap = 'round';
    ctx.lineJoin = 'round';
    ctx.setLineDash([8, 4]);
    ctx.beginPath();
    ctx.moveTo(sel.lassoPoints[0].x, sel.lassoPoints[0].y);
    for (let i = 1; i < sel.lassoPoints.length; i++) {
      ctx.lineTo(sel.lassoPoints[i].x, sel.lassoPoints[i].y);
    }
    ctx.stroke();
    ctx.setLineDash([]);
    ctx.restore();
  }

  // 2. 拖拽预览：绘制偏移中的笔画
  if (sel.dragging && sel.dragSnapshot) {
    for (const s of sel.dragSnapshot) {
      drawBlackboardStroke(ctx, s);
    }
    // 绘制拖拽中的包围盒
    const bb = computeBBox(sel.dragSnapshot);
    if (bb) drawSelectionBBox(ctx, bb);
    return;
  }

  // 3. 已确认的选中笔画高亮
  if (sel.confirmedIndices.length > 0 && page) {
```

```
    // 对被选中的笔画绘制光晕
    for (const i of sel.confirmedIndices) {
      const s = page.strokes[i];
      if (!s || s.points.length === 0) continue;
      ctx.save();
      // 外层光晕
      ctx.strokeStyle = 'rgba(56, 189, 248, 0.25)';
      ctx.lineWidth = s.width + 8;
      ctx.lineCap = 'round';
      ctx.lineJoin = 'round';
      ctx.beginPath();
      ctx.moveTo(s.points[0].x, s.points[0].y);
      for (let j = 1; j < s.points.length; j++) {
        ctx.lineTo(s.points[j].x, s.points[j].y);
      }
      ctx.stroke();
      ctx.restore();
    }
    // 绘制包围盒
    if (sel.bbox) drawSelectionBBox(ctx, sel.bbox);
  }
}

// 绘制选中包围盒
function drawSelectionBBox(ctx, bbox) {
  const pad = 8;
  const w = bbox.maxX - bbox.minX + pad * 2;
  const h = bbox.maxY - bbox.minY + pad * 2;
  ctx.save();
  ctx.strokeStyle = 'rgba(56, 189, 248, 0.8)';
  ctx.lineWidth = 1.5;
  ctx.setLineDash([6, 3]);
  ctx.strokeRect(bbox.minX - pad, bbox.minY - pad, w, h);

  // 四个角拖拽手柄
  ctx.setLineDash([]);
  ctx.fillStyle = 'rgba(56, 189, 248, 0.9)';
  const handleSize = 6;
  const corners = [
    { x: bbox.minX - pad, y: bbox.minY - pad },
    { x: bbox.maxX + pad, y: bbox.minY - pad },
    { x: bbox.minX - pad, y: bbox.maxY + pad },
    { x: bbox.maxX + pad, y: bbox.maxY + pad },
  ];
  for (const c of corners) {
    ctx.fillRect(c.x - handleSize / 2, c.y - handleSize / 2, handleSize, handleSize);
  }
  ctx.restore();
}
```